第 7 章

Systems Programming: Designing and Developing Distributed Applications

案例研究：融会贯通

7.1 基本原理和概述

本章的目的是提供两个完整、详细的案例研究，把本书前面章节讲到的理论和实践中的知识点放到应用场景中。这两个新案例，加上多玩家分布式游戏案例研究，共同作为主线，贯穿了本书前面各章，用于整合 4 个视角（进程、通信、资源和体系结构），通过展示一系列不同的生动的例子，给出了分布式应用程序完整的全局概览。

两个案例是：1）时间服务客户端，2）事件通知服务。选定这两个案例，旨在集成地提供对前述章节中讨论的概念的全面应用。每个案例研究都提供了独有的机会，来展示特定结构、行为特性、技术，以克服前面章节中指出的分布式应用程序设计开发方面面临的挑战。

对于每一个案例，它们的设计和开发过程都保留了带注释的详细文档，以帮助读者跟得上不同的步骤，并理解其中可行的选择，以及最后采取的决策。本书提供了完整的程序源码。

7.2 用例说明

第一个案例研究是一个基于网络时间协议（Network Time Protocol，NTP）服务的客户端侧的应用程序。NTP 服务由美国国家标准与技术研究院（National Institute of Standards and Technology，NIST）提供。NTP 在第 6 章讲述过。

本案例研究提供了一个有趣的例子情形：开发者必须设计一个应用程序，与带有预定义接口的已有服务集成。在这种情形下，开发者必须确保与预先发布的接口规范严格一致。对于 NTP 用例，其通信协议以固定的消息格式在请求 - 应答基础上运行。NTP 协议的请求 - 应答性质隐含地定义了交互机制。

这个案例研究还演示了使用 DNS 解析器来定位请求的 NTP 服务器实例的 IP 地址。这是一个重构的例子：创建一个软件库，使逻辑模块化，也方便代码重用；创建两个不同的用户前端组件，修改用户界面，但使用相同的库方法，从而有相同的底层功能。

第二个案例研究是一个事件通知服务（Event Notification Service，ENS），由一组应用程序组件构成。这些组件用作 ENS 的发布者客户端和消费者客户端。ENS 解耦了应用组件（发布者组件和消费者组件），使得它们不用直接通信。ENS 不需要知道哪些事件将要发布，以及这些事件的含义。它只是存储事件的“类型 - 值”对。当发布了一个新的事件类型，或这发布了一个已有事件类型的新值时，ENS 更新其事件的“类型 - 值”表。事件值被推送到已订阅了该事件类型（注册了对此事件感兴趣）的所有客户端。作为一种服务，ENS 可以被多个不同的应用程序并发调用。它不必知道哪些应用程序正在使用它，或者这些应用程序的目的是什么。

475 ~ 476

使用 ENS 会在设计阶段受益。消费者组件不需要知道未来产生事件数据的发布者的标识，反之亦然。这些组件对其他特定组件将不存在固有的依赖关系，从这种意义上讲，组件的设计因此变得更简单、更快捷和更安全。ENS 提供了运行时的灵活性，并按照组件间的发

- 规模可扩展和高效率。ENS 必须有效地使用资源。特别地，通信强度必须低；事件更新应仅发送给已订阅该事件的客户端子集。
- 可用性。使用的模式假设 ENS 服务器长期连续运行。这要求它是健壮的。

7.4.3 体系结构和代码结构

ENS 和 DS 属于在第 6 章讨论过的公共服务范畴。如图 7-17 的交互图所示，综合使用这些服务，有助于应用程序组件之间的完全解耦。应用程序组件之间没有直接通信。

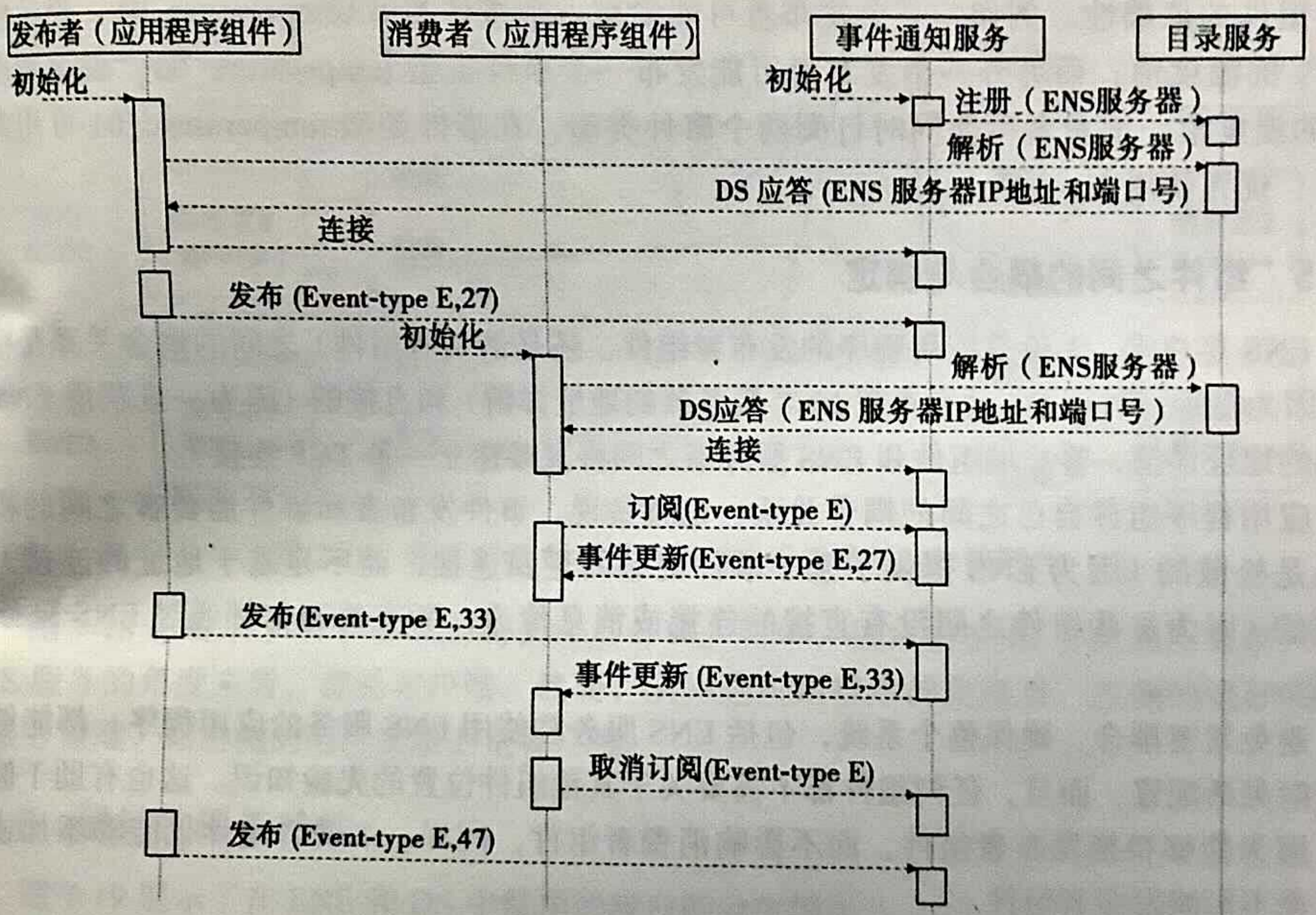


图 7-17　ENS 交互的时序图表示

图 7-17 展示了一个典型的交互过程，其中包括 ENS、DS 和一对儿应用程序组件：一个发布者组件和一个消费者组件。在该时序图中，ENS 首先向 DS 注册。然后，发布者组件启动，获得了来自 DS 的 ENS 的地址详情，并使用该信息连接 ENS。随后，发布者发布了一个值为“27”的新事件类型“E”。消费者组件随后启动了，也从 DS 获取 ENS 的地址，并同样与 ENS 建立连接。消费者订阅事件类型“E”，同时，ENS 发回缓存的对应该事件类型的值“27”。然后，发布者发布了事件类型“E”的新值“33”。ENS 响应，将该事件类型的新值推送给了消费者。接下来，消费者取消了对事件类型“E”的订阅。随后，发布者发布事件类型“E”的新值为“47”。ENS 不再将事件类型的新值推送给消费者。

494

7.4.4 关注点的分离

ENS 设计用来支持分布式应用，其中包括两类组件（发布者组件和消费者组件）。从 ENS 系统角度看，这两类组件可视为客户端。从应用程序本身内部看，发布者可能实际上在服务器侧，消费者实际上在客户端侧。对于特定事件类型，它们行为依赖于特定的应用程序，就这一点而言，功能上的关注点和行为上的关注点就自然地分开了。然而，当一个组件

可能发生的问题例如有：NTP 服务器崩溃、使用的服务域名不正确而无法解析为 NIST 服务器的 IP 地址、使用 UDP 协议传输的 NTP 请求消息丢失或破坏、NTP 服务器的响应消息丢失或破坏。

- 模块化架构。有必要支持将时间服务客户端的行为整合到需要精确时间戳的各种应用程序中。这意味着，核心 NTP 客户端功能应该以库的形式部署，同时，还有必要能够建立用于测试和评估库功能的简单独立的 NTP 客户端应用程序。
- 高效率。NTP 客户端逻辑应该尽可能使用最低复杂度的代码实现，仅有必需的功能，并且，在处理时间和网络带宽方面，尽可能少地占用资源。因为 NTP 客户端将作为一个库模块来开发，所以这点非常重要。库模块接口必须尽可能简单，仅暴露最小的一组方法，用于模块配置和发起 NTP 时间戳请求。

7.3.3 体系结构和代码结构

应用程序的设计围绕负责处理核心行为的库组件展开。核心行为包括：配置本地通信资源（也就是说，创建套接字并配置其在非阻塞模式下运行），将 NIST 时间服务器域名解析为 IP 地址（这需要使用 DNS 解析器），与该 IP 地址上的 NTP 服务器实例建立通信。代码库中执行发送 NTP 请求和接收 NTP 响应的通信方法，必须总是在较短的时间窗口内返回结果，甚至，在 NTP 服务器不响应的情况下，库模块必须继续运行，一定不能崩溃或冻结。代码库的逻辑保持了尽可能简单，仅包含一个类，如图 7-1 所示。

CNTP_Client
m_TimeService_SockAddr : SOCKADDR_IN m_NTP_Timestamp : NTP_Timestamp_Data
CNTP_Client(pbSocketOpenSuccess : bool*) ResolveURLtoIPaddress(cStrURL : char*) : SOCKADDR_IN Get_NTP_Timestamp(pNTP_Timestamp : NTP_Timestamp_Data*) : unsigned int Send_TimeService_Request() : bool Receive(pNTP_Timestamp : NTP_Timestamp_Data*) : bool

图 7-1 CNTP_Client 类（NTP 客户端库的核心代码）

图 7-1 展示了 CNTP_Client 类模板。当调用构造函数 CNTP_Client() 时，NTP 客户端库组件初始化。这将初始化套接字，用于与 NTP 服务器通信。其中包括两个内部状态变量。一是特定 NTP 服务器的 IP 地址，在应用程序组件调用 ResolveURLtoIPaddress() 方法时，使用一个 NIST 服务器的 URL 作为传入参数，就设置了这个参数的状态变量。二是时间戳数据结构 NTP_Timestamp_Data（见图 7-2）。它根据接收到的 NTP 时间戳，在代码库中创建。时间戳的值，使用自 1900 年起始的秒数作为原始格式编码。通过调用 Get_NTP_Timestamp() 方法，作为正在发起时间戳请求的应用程序组件的结果，时间戳数据结构从库中返回。Send_TimeService:Request() 和 Receive() 方法用于在内部构建通信逻辑，处理 NTP 服务器相连接的库。这两个方法都没有暴露在代码库的应用程序组件接口中。

478

本案例研究包括两个应用程序组件。每个组件都能结合 NTP 客户端库组件，建立定制的时间服务客户端应用程序。这两个应用程序组件，在其提供的用户界面方面有所不同，但它们在内部使用库功能的方式相同。与 NTP 服务的外部连接由库组件管理，而不是应用程序组件。因此，使得两个应用程序在这方面是一致的。

```
struct NTP_Timestamp_Data {
  unsigned long lUnixTime;  // Seconds since 1970 (secsSince1900 - Seventy Years)
  unsigned long lHour;
  unsigned long lMinute;
  unsigned long lSecond;
  };
```

图 7-2　代码库中的 Get_NTP_Timestamp() 方法返回的 NTP_Timestamp_Data 数据结构

第一个应用组件提供了一个图形用户界面（GUI），见图 7-3。该应用程序组件的核心类模板如图 7-4 所示。

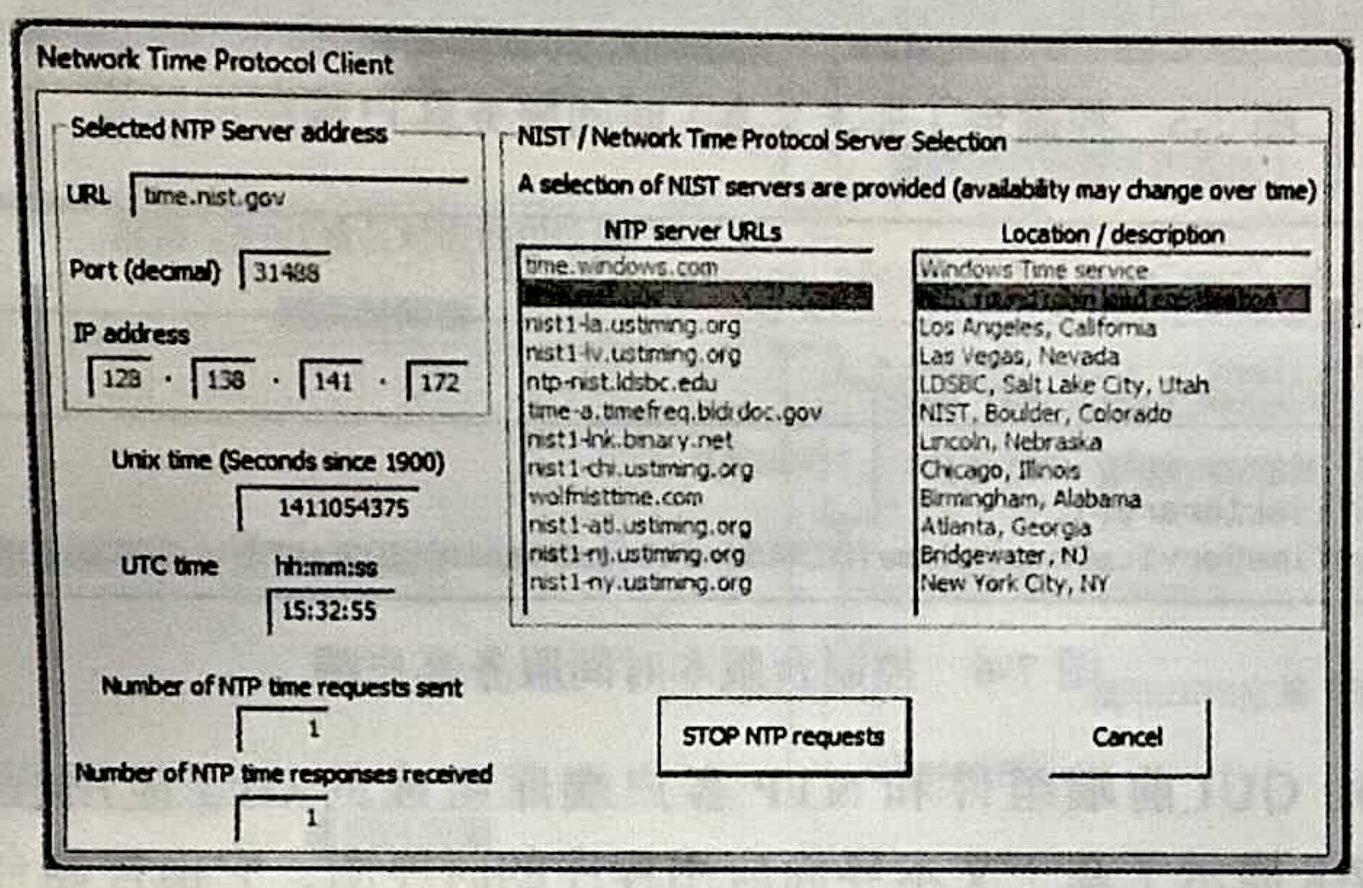


图 7-3　GUI 时间服务客户端用户界面

```
CCaseStudy_TimeServiceClient_GUIDlg
m_pNTP_Client : CNTP_Client*
m_NTP_Timestamp : NTP_Timestamp_Data
CCaseStudy_TimeServiceClient_GUIDlg(pParent : CWnd*)
OnLbnSelchangeNtpServerUrlList()
OnTimer(nIDEvent : UINT)
```

图 7-4　CCaseStudy_TimeServiceClient_GUIDlg 类

图 7-4 显示了 CCaseStudy_TimeServiceClient_GUIDlg 类模板。这是 GUI 应用程序组件的核心类，仅包含与整合库组件相关的关键细节，省略了用户特定界面细节，如控件（按钮、列表框等）。m_pNTP_Client 变量是一个指向库模块实例的指针，在运行构造方法 CCaseStudy_TimeServiceClient_GUIDlg() 时初始化。每当用户从提供的 URL 列表（见图 7-3）中选择一个 NIST 服务器域名时，OnLbnSelchangeNtpServerUrlLis() 事件处理器方法就被调用。该方法把域名传递到库，再将域名解析为对应的 IP 地址。通过 NTP 客户端库，OnTimer 事件处理器方法每隔 5 秒发送一次 NTP 时间请求。发送的时间间隔由一个可编程定时器控制。 479

第二个应用组件，提供了文本的用户界面，见图 7-5。这个过程式应用程序组件以类模板模样的格式展示在图 7-6 中。

图 7-6 中，按照它们基于 NTP 客户端库的界面，展示了控制台时间服务客户端的用户界面组件的关键特性。m_pNTP_Client 变量是指向库模块实例的指针，在 InitialiseLibrary() 方法中初始化。ResolveTimeServiceTOIPAddress() 方法传递用户选择的 NIST 时间服务器的 URL 给库。通过 _tmain() 方法，NTP 时间请求每间隔 5 秒发送一次（借助 NTP 客户端库）。

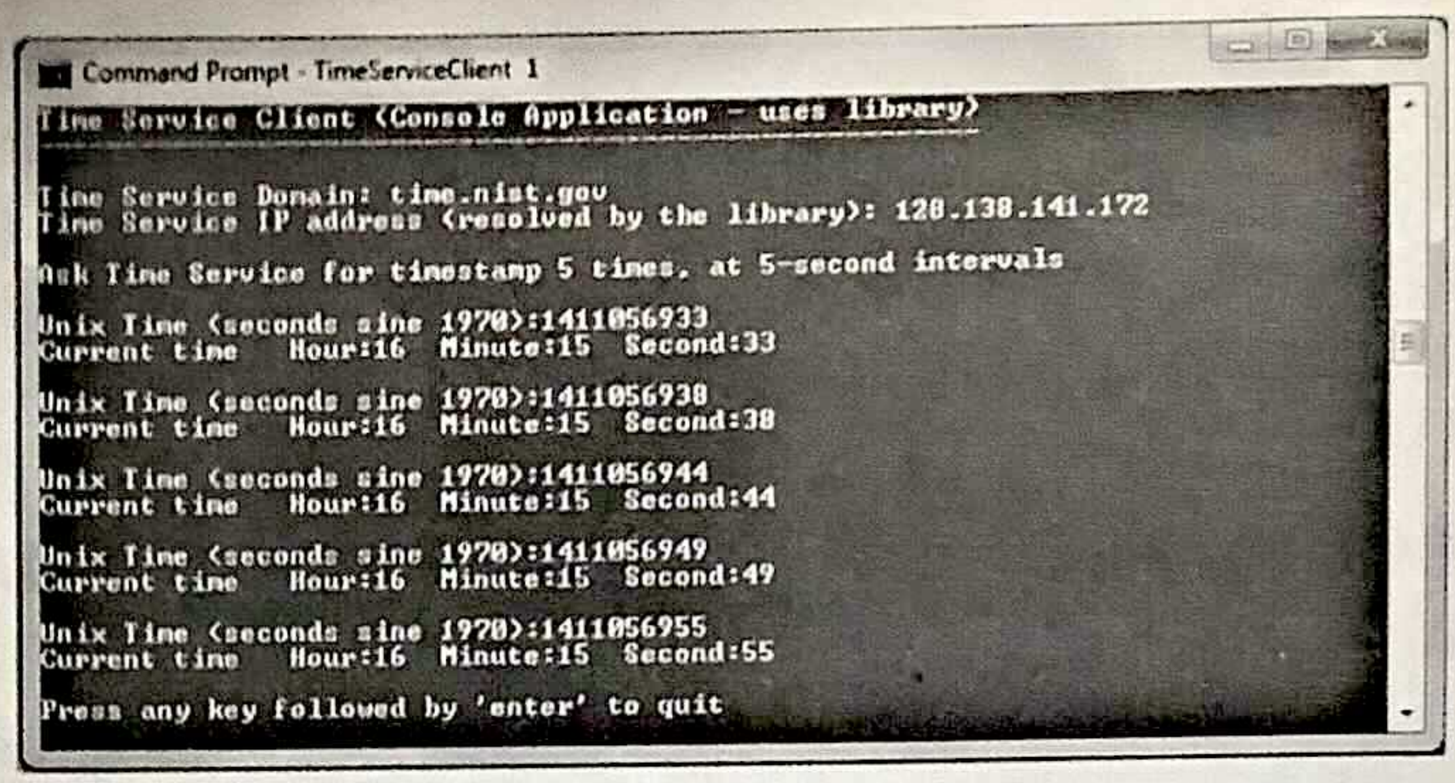


图 7-5 控制台（基于文本）时间服务客户端用户界面

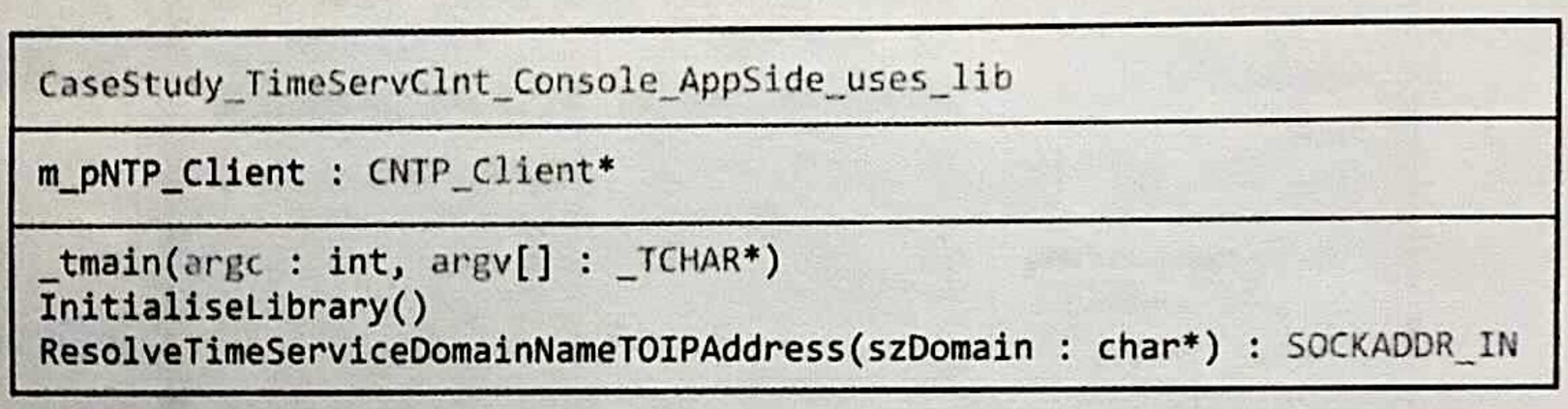


图 7-6 控制台版本时间服务客户端

图 7-7 展示了由 GUI 前端组件和 NTP 客户端库组成的 GUI 应用程序的时序图。与图 7-8 相对比，图 7-8 展示了基于文本界面应用程序的时序图，它由控制台前端组件和 NTP 客户端库组成。

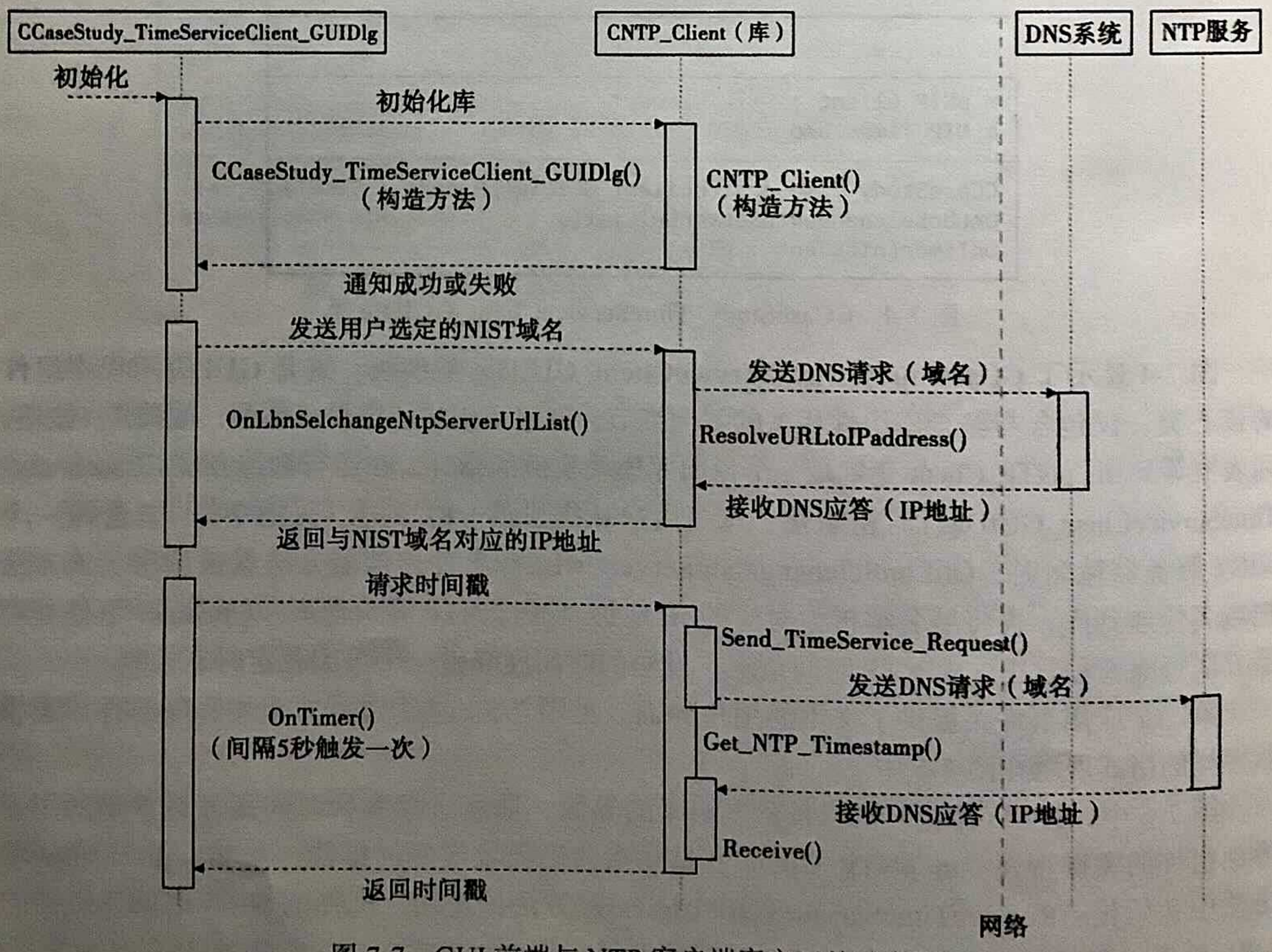
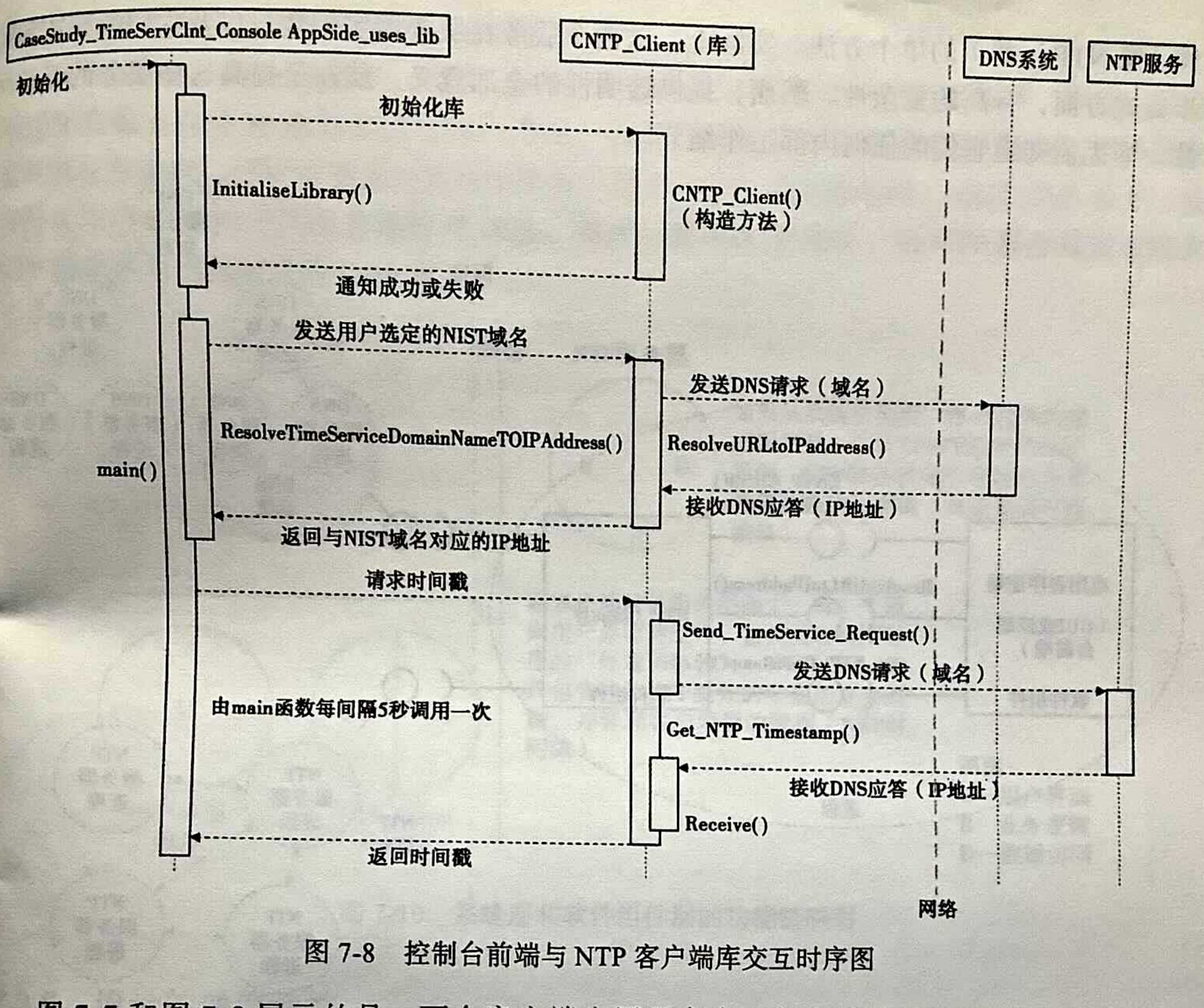


图 7-7 GUI 前端与 NTP 客户端库交互的时序图



480～481

图 7-8　控制台前端与 NTP 客户端库交互时序图

图 7-7 和图 7-8 展示的是，两个客户端应用程序中，每个前端和 NTP 客户端库之间交互的情形。两个前端应用程序在其代码结构和内部行为方面是不同的。最显著的区别是，基于 GUI 的前端是事件驱动的（也就是说，它的行为由事件驱动，比如用户的鼠标点击或键盘输入，也可能是定时器驱动的事件），而控制台应用程序是程序性的（也就是说，它起始于一个主函数，自此其他函数按照顺序被调用，以实现程序的逻辑）。尽管在两类前端终端之间存在差异，但仍然可以从图表中清晰地看出，这两种情况下使用库的方式是相同的：

- 该库首先被初始化（调用构造方法）。初始化过程包括：建立通信套接字（将用于向 NTP 服务器发送请求）和接收来自 NTP 服务器的响应。该套接字配置成为非阻塞工作模式。
- 该库在 ResolveURLtoIPaddress() 方法中解析 NIST 服务器的域名。它从内部使用 gethostbyname（一个 DNS 解析器）自动连接到外部 DNS 服务（DNS 如何工作的细节见第 6 章）。
- 该库通过 Get_NTP_Timestamp() 方法向外部 NTP 服务发送 NTP 时间戳请求。

时序图也阐明了透明性的一个重要特性，用到的两个外部服务（DNS 和 NTP）都分别描述成单个对象，来响应请求。这是为开发者提供的外部视图，尽管每个服务的内部都十分复杂。图 7-9 提供的组件图进一步强调了透明性。

482

图 7-9 的左半部分展示了客户端应用程序如何由两个软件组件构成：一个前端用户界面和 NTP 客户端库，处理了与使用的两个外部服务（DNS 和 NTP）的全部外部连接和通信。该图的右侧还将外部服务描述为虚拟组件，本质上，整个 DNS 服务抽象为一个单独组

件（像 NTP 一样）的单个方法。实际上，这两个服务在服务器实例数目以及它们的互连和同步方式方面，有高的复杂性。然而，提供透明性的全部意义，就在于提高这些服务的可访问性，而无需知道它们的任何内部工作细节。

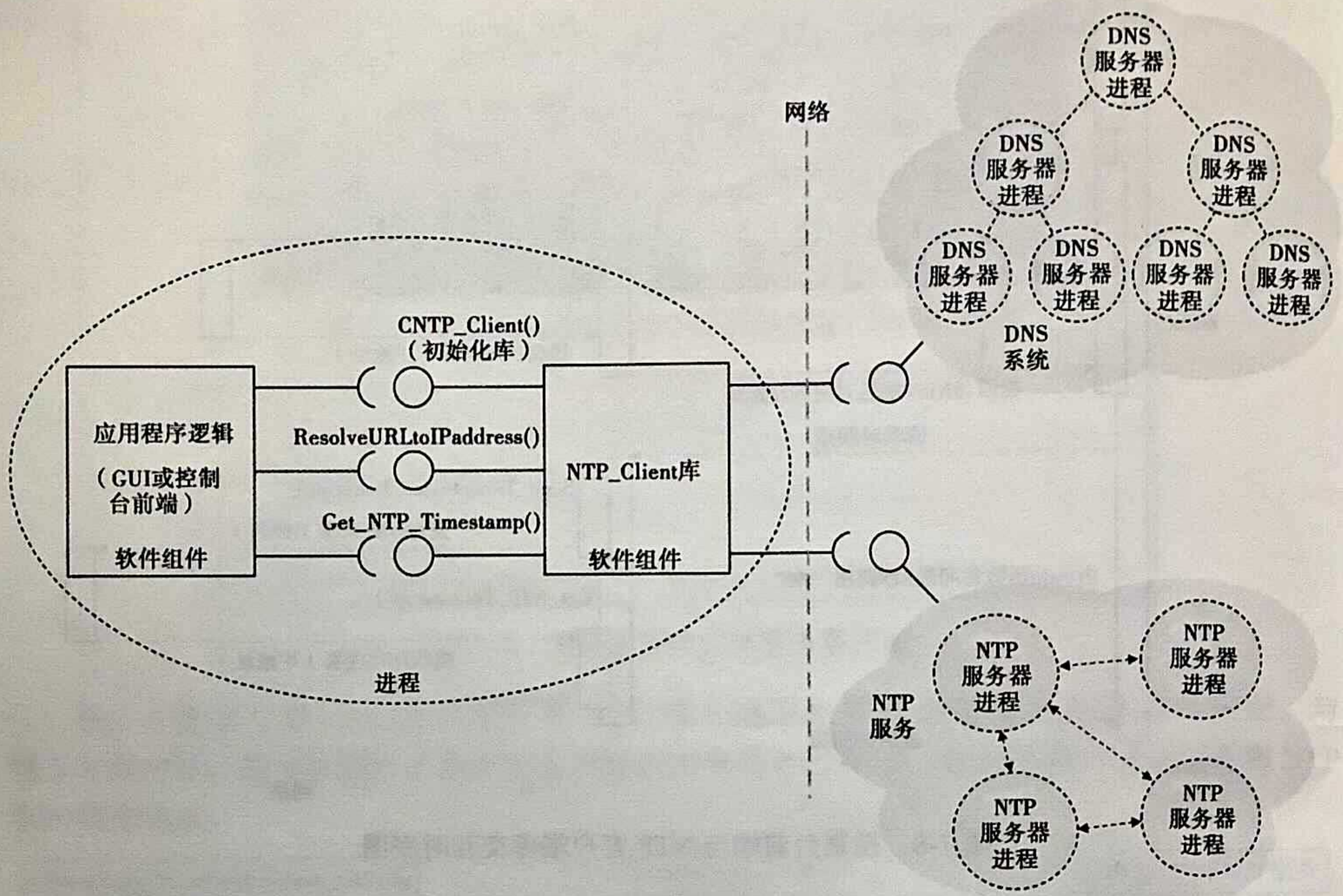


图 7-9　组件图说明内部和外部的连接过程

7.3.4　关注点的分离

沿组件边界仔细划分关注点，有助于设计的清晰化和实现的简单化（尤其是在减少软件组件间的耦合程度和交互性方面）。这也有利于功能的模块化重用，以及能够简化测试。

NTP 服务提供了一个清晰定义的功能。这里提供的用例场景中，客户端侧是一个用户界面前端，其功能仅用于显示由 NTP 服务提供的时间戳值。然而，客户端侧应用程序的功能可以分解得更精细。应用程序包括两个软件组件：其一，提供了用户界面；另一个，（NTP_Client 库）提供了必要的业务逻辑，以定位和连接到 NTP 服务，发送 NTP 请求，以及接收

483 和解释 NTP 服务响应。图 7-10 把分离关注点的双层视图放在了同一个示意图中。

图 7-10 展示了不同的功能链之间的映射，取决于这些功能所应用的层次。在最高层，NTP 客户端应用程序只提供了用户界面功能，但在内部，全部逻辑分解到两个组件中。在进一步细分的层面上，我们可以把纯粹的用户界面相关的逻辑，从管理与外部服务通信的业务逻辑和管理数据资源（本案例指时间戳）的业务逻辑中分离出来。

7.3.5　组件之间的耦合与绑定

NTP 客户端和 NTP 服务之间的耦合关系可以称为松散的（因为客户端能够根据运行时提供的 NIST 服务域名连接到任意的 NTP 服务实例）和直接的（因为一旦确定了服务实例，

客户端和服务器组件之间就会发生直接通信）。

客户端绑定到时间服务器。NIST 拥有一批以熟知域名部署的时间服务器。但是，这并不意味着服务器的 IP 地址是固定的。因此，选定的域名必须能解析为 IP 地址，而不是硬编码或长期缓存。绑定能够发生在两个阶段。首先，用户选择的 URL，通过 DNS 服务，被解析成相应的 NTP 时间服务器的 IP 地址；然后，利用该 IP 地址，向 NTP 服务器发送包含 NTP 请求消息的点对点的 UDP 报文段。

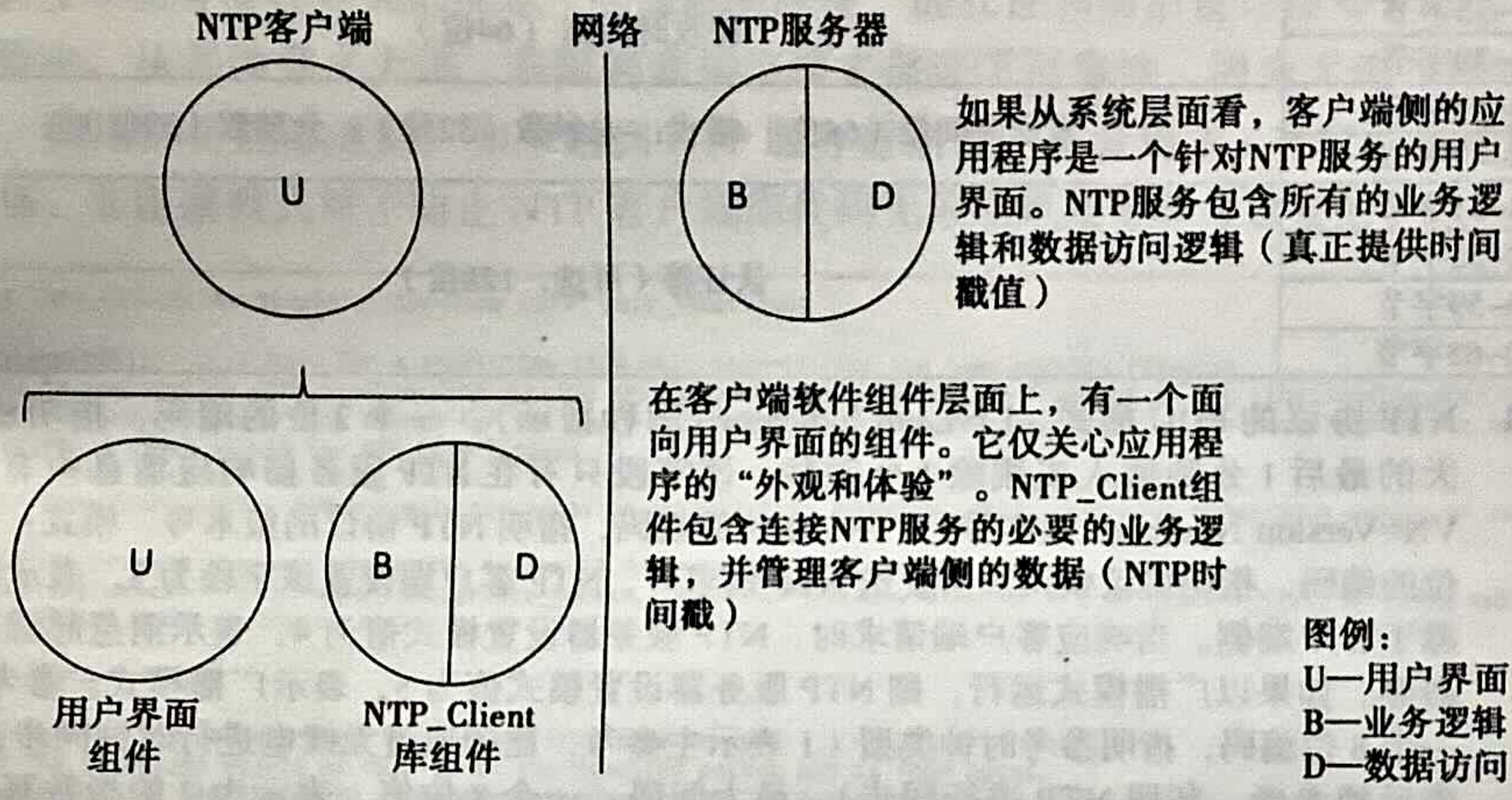


图 7-10　系统层和软件组件层的功能链映射

7.3.6　设计的通信特性

NTP 使用请求 – 应答协议。它使用一个固定的熟知端口号 123 来识别特定的计算机上的 NTP 服务器进程。宿主计算机以其 IP 地址标识（反过来从一个 NIST 公布的域名得到，请参阅前节内容）。 484

1. 消息类型和语义

有两种消息类型：NTP 请求和 NTP 响应。只有当 NTP 请求发送到 NTP 服务器时，它才是有效的。同样，只有当一条来自 NTP 服务的 NTP 响应作为对早些发送的 NTP 请求的响应时，它才是有效的。通信的语义如此简单。

2. NTP 协议定义单元（PDU）

NTP 请求消息和 NTP 响应消息都编码为固定大小的 64 字节数组。NTP PDU 格式，采用了一个结构体替代了线性数组。这个结构体把多个字节聚合成不同的固定长度的字段，从而描述出消息内容的应用含义。图 7-11 展示了 NTP PDU，定义了 NTP 请求和响应消息的格式。

NTP 请求消息的填充方法如下：整个 48 字节的数组置零，然后，LI、VN 和模式字段的值分别设置为 3、4、3，这表明，消息是一个 NTP 版本 4 的消息，从客户端发来（即请求消息），带有一个当前未同步的时钟值（也就是说，消息中的时间戳值是没有意义的），见图 7-12。

	0~7位	8~15位	16~23位	24~31位
0~3 字节	LI | VN | 模式	参考时钟（8位）	最大间隔（8位）	精度（8位）
4~7 字节	根延迟（32位）			
8~11 字节	根分散（32位）			
12~15 字节	参考标识符（32位）			
16~19 字节 20~23字节	参考时间戳（64位）			
24~27字节 28~31字节	初始时间戳（64位）			
32~35字节 36~39字节	接收时间戳（64位）			
40~43字节 44~47字节	发送时间戳（64位）格式：-总秒数（32位）：分秒数（32位）			
48~51字节 52~55字节 56~59字节 60~63字节	认证符（可选，128位）			

图 7-11 NTP 协议的 PDU 格式。LI=Leap Indicator（闰秒指示）：一个 2 位的编码，指明将在当天的最后 1 分钟插入或删除 1 个闰秒。该字段只有在 NTP 服务器响应消息中有意义。VN=Version Number（版本号）：一个 3 位的编码，指明 NTP 协议的版本号。模式：一个 3 位的编码，指明协议模式。当发送 NTP 请求时，NTP 客户端设置该字段为 3，表示消息起源于客户端侧。当响应客户端请求时，NTP 服务器设置模式值为 4，表示消息起源于服务器侧。如果以广播模式运行，则 NTP 服务器设置模式值为 5，表示广播模式。参考时钟：一个 8 位编码，指明参考时钟类型（1 表示主参考，比如通过无线电进行时钟同步；2～15 表示辅参考，利用 NTP 进行同步）。最大间隔：一个 8 位值，表示为 2 的指数幂，指明连续的消息之间的最大时间间隔。该字段的取值范围为 4～17，意思是说，该时间间隔的取值为 16 秒、32 秒、64 秒、128 秒，……，直到 131 072 秒（大约为 36 小时）。精度：一个 8 位的编码，代表时钟的精度，表示为 2 的指数幂，取值范围为 -20～-6，意味着精度值为 64 分之 1 秒，或者更优。根延迟：一个 32 位的值，指明到主参考时钟源，以秒为单位的往返延迟。根分散：一个 32 位的无符号值，指明最大误差，可以最大到几百毫秒。参考标识符：一个 32 位的值，标识特定的参考时钟源。对于第 1 层（主服务器），该值是一个 4 字符的编码；而对于辅服务器，该值是同步时钟源的 IPv4 地址。参考时间戳：最近一次设置或更正系统时钟的时间。初始时间戳：请求消息从客户端发送（往服务器）的时间。接收时间戳：请求消息到达服务器的时间（或应答消息到达客户端的时间，取决于消息的方向）。发送时间戳：应答消息从服务器发送（往客户端）的时间。认证符：一个可选的值，用于 NTP 验证。

485

```
bool CNTP_Client ::Send_TimeService_Request()
{
    memset(SendPacketBuffer, 0,   NTP_PACKET_SIZE ); // Zero -out entire 48 -byte array
    // Initialize values needed to form NTP request
    SendPacketBuffer[0] = 0xE3;   // 0b11100011;
                                  // LI bits 7,6           = 3 (Clock not synchronised),
                                  // Version bits 5,4,3    = 4 (The current version of NTP)
                                  // Mode bits 2,1,0       = 3 (Sent by c lient)

    m_iSendLen =  sizeof (SendPacketBuffer);

    int iBytesSent = sendto(m_TimeService_Socket, (char FAR *)SendPacketBuffer, m_iSendLen, 0,
         (const struct sockaddr FAR *)&m_TimeService_SockAddr, sizeof (m_TimeService_SockAddr));
    if(INVALID_SOCKET == iByt esSent)
    {
       return false;
    }
    return true;
}
```

图 7-12 Send_TimeService：库中的 CNTP_Client 类的请求方法

图 7-12 展示了在时间服务客户端库组件中，设置 NTP 请求消息内容并发送消息的程序代码。仅有第 1 字节的配置用来通知接收者，该消息的类型是一个请求（来自客户端），遵循了 NTP 版本 4 的格式。

图 7-13 展示了库组件的接收方法。当 NTP 请求消息发送（基于 UDP）给 NTP 时间服务器后，调用这个方法，开始等待 NTP 响应（也基于 UDP）。一个短时间延迟和单次调用 recvfrom（也就是说，不用在循环中周期性重复）的结合使用，加上把套接字配置成非阻塞模式，就在 3 项有潜在冲突的需求（低延迟响应性、健壮性和简洁设计性等）之间达成了一个有用折中。从某种意义上讲，非阻塞套接字模式保证了可靠性，因为无论 NTP 服务器是否响应，调用都将能够返回。如果遇到 NTP 服务器崩溃，或者请求 / 响应消息在网络中丢失或损坏，非阻塞模式对于防止 NTP 客户端库代码无限期阻塞是非常重要的。 486

```
bool CNTP_Client::Receive(NTP_Timestamp_Data* pNTP_Timestamp)
{
   Sleep(500);      // Wait for a short time (500 milliseconds) for the time service response.
   // In combination with non-blocking receive this prevents the application freezing if the time service does not
   // respond but waits long enough for the reply RTT so mostly avoids missing an actual reply and avoids the need
   // for a timer within the NTP_Client class
   // Tested with the following values:
   // 100ms(unreliable) 200ms(highly dependent on network RTT) 400ms(generally reliable) 500ms(adds margin of safety)

   // The process inspects its buffer to see if any messages have arrived
   int iBytesRecd = recvfrom(m_TimeService_Socket, (char FAR*)ReceivePacketBuffer, NTP_PACKET_SIZE, 0, NULL, NULL);
   if(SOCKET_ERROR == iBytesRecd)
   {
      return false;
   }
   // Receive succeeded (response received from Time server)
   // The timestamp starts at byte 40 of the received packet and is four bytes,
   unsigned long secsSince1900 = (ReceivePacketBuffer[40] << 24) + (ReceivePacketBuffer[41] << 16) +
                                 (ReceivePacketBuffer[42] << 8) + ReceivePacketBuffer[43];
   const unsigned long seventyYears = 2208988800UL;              // Unix time starts on Jan 1 1970 (2208988800 seconds)
   pNTP_Timestamp->lUnixTime = secsSince1900 - seventyYears;   // Subtract seventy years:
   pNTP_Timestamp->lHour = (pNTP_Timestamp->lUnixTime  % 86400L) / 3600;
   pNTP_Timestamp->lMinute = (pNTP_Timestamp->lUnixTime  % 3600) / 60;
   pNTP_Timestamp->lSecond = pNTP_Timestamp->lUnixTime  % 60;
   return true;
}
```

图 7-13　库中的 CNTP_Client 类的接收方法

使用了 500ms 的延迟，考虑了从发送请求消息到接收响应的往返延迟（Round-trip Time，RTT）。网络延迟在不断变化中，因此，对于远程传输网络（比如，在连接 NTP 时间服务器的案例中），从来找不到完美的固定的超时值。通过实验发现，500ms 的延迟值是一个非常好的折中。一方面，这个等待时间足够长，从而保证了近乎所有情形下的 NTP 响应都能收到；另一方面，又没有增加太多的额外延迟。即便是 RTT 接近 0，该方法也仅仅插入了半秒的额外延迟。

时间戳填充在响应消息的第 40～47 字节（发送时间戳字段）。时间戳值占了 64 位的宽度，其中的高 32 位表示整秒数，低 32 位表示秒的小数部分。对于案例研究应用程序来说，仅考虑时间戳的整数部分已经足够（因此，第 40～43 字节的值的用法，可参见如图 7-13 中的代码）。在需要更为精准的时间戳的应用中，也可以考虑使用时间戳的小数部分。

3. DNS 解析 URL

gethostbyname DNS 解析器已内置于库代码中，从而，将能够自动连接 DNS 系统，解析 NIST 时间服务器的域名为 IP 地址，见图 7-14。

4. 选择通信设计方案的理论依据

选择了固定延迟的接收机制，以保持代码简单，同时，确保响应可预测，并防止调用在发生错误时被阻塞。该机制设计和操作都简单：使用了一个短时延，配合了一个非阻塞的套

接字调用。

然而，如果在有些应用中，也采用500毫秒的固定延迟，却出了问题，那么还可以选择其他方法。

```
SOCKADDR_IN CNTP_Client::ResolveURLtoIPaddress(char* cStrURL)
{
    DWORD dwError;
    struct hostent *TimeServiceHost;
   TimeServiceHost = gethostbyname((const char*)cStrURL);

    if(NULL == TimeServiceHost)
    { // gethostbyname failed to resove time service domain name
      dwError = WSAGetLastError();
      m_TimeService_SockAddr.sin_addr.S_un.S_un_b.s_b1 = 0;
      m_TimeService_SockAddr.sin_addr.S_un.S_un_b.s_b2 = 0;
      m_TimeService_SockAddr.sin_addr.S_un.S_un_b.s_b3 = 0;
      m_TimeService_SockAddr.sin_addr.S_un.S_un_b.s_b4 = 0;
      m_bTimeServiceAddressSet = false;
    }
    else
    { // gethostbyname successfully resoved time service domain name
      m_TimeService_SockAddr.sin_addr.S_un.S_addr = *(u_long *) TimeServiceHost->h_addr_list[0];
                                                  // Get first address from host's list of addresses
      m_TimeService_SockAddr.sin_family = AF_INET;       // Set the address type
      m_TimeService_SockAddr.sin_port = htons(NTP_Port); // Set the NTP port (Decimal 123)
      m_bTimeServiceAddressSet = true;
    }
   return m_TimeService_SockAddr;
}
```

487

图7-14 库中CNTP_Client类的ResolveURLtoIPaddress方法

可以构建一个自我调整系统。其中，开始先尝试设置50毫秒的短延迟，如果recvfrom超时，则延迟翻倍，并重新发送请求消息。持续翻倍，直到未超时收到NTP响应（从而，一直自动调整客户端的RTT等待时间）。这种方法存在3个问题。1）增加了复杂度。2）NTP客户端发送请求的频率不能以高于每4秒一次（调用者以更高的频率发送请求，会解释为是对NTP服务的拒绝服务攻击）。为了满足这一要求，自我调整的方法将不得不在两次尝试之间等待4秒。这样会导致在收到时间戳值之前，引入了明显的延迟（尽管时间戳自身的延迟有可能比500毫秒的固定延迟还要小）。3）网络延迟在不断变化，因此，即使自我校正完成了，RTT仍然有可能会变化。

提高客户端侧响应能力的另一种方式是，以更短的时间间隔（例如50毫秒），发起非阻塞状态下的recvfrom调用，可以由可编程的定时器机制驱动。在NTP服务器没有响应的情况下，这种方式需要设置一个停止条件，也就是必须确定一个临界点，比如调用10次之后（也就是说，保持延迟的上限为500毫秒）就自动停止。与案例中采用的设计方案相比，这种方法提高了复杂度，还占用了更多的运行时资源。

7.3.7 实现

时间服务客户端应用程序已经分3个阶段实现，其中每个阶段都可以用作完整的示例代码例子。

第一阶段是开发一套完整的应用程序，包含完整的NTP客户端功能，集成用户界面逻辑。这种方法代表了一种基于原型的快速开发模式。就像在本案例中一样，它仅能用于功能很少的应用程序。第一阶段的完整项目是CaseStudy_TimeServiceClient_GUI Phase1 Monolithic。

第二阶段是代码重构。将NTP客户端的业务逻辑放到与用户界面功能相关的独立类。

第二阶段重构的项目是 CaseStudy_TimeServiceClient_GUI Phase2 Refactored。

第三阶段是创建一个包含 NTP 客户端侧功能的库，再通过把库嵌入应用程序中，重用这些功能。第三阶段的库项目，是 CaseStudy_TimeServiceClient Phase3 Library。

为了演示库方法的好处，开发了两个独立的前端应用程序。第一个前端应用程序提供了与项目的第一阶段和第二阶段相同的 GUI 界面。第二个前端应用程序提供了一个基于文本的界面，因为使用了库，它们的时间服务相关的功能完全相同，但界面完全不同。第三阶段（a 部分）GUI 应用程序项目是 CaseStudy_TimeServiceClient_GUI Phase3App-side uses library。第三阶段（b 部分）基于文本的应用程序项目是 CaseStudy_TimeServClnt_Console AppSide uses lib。

图 7-15 展示了实现路线图。

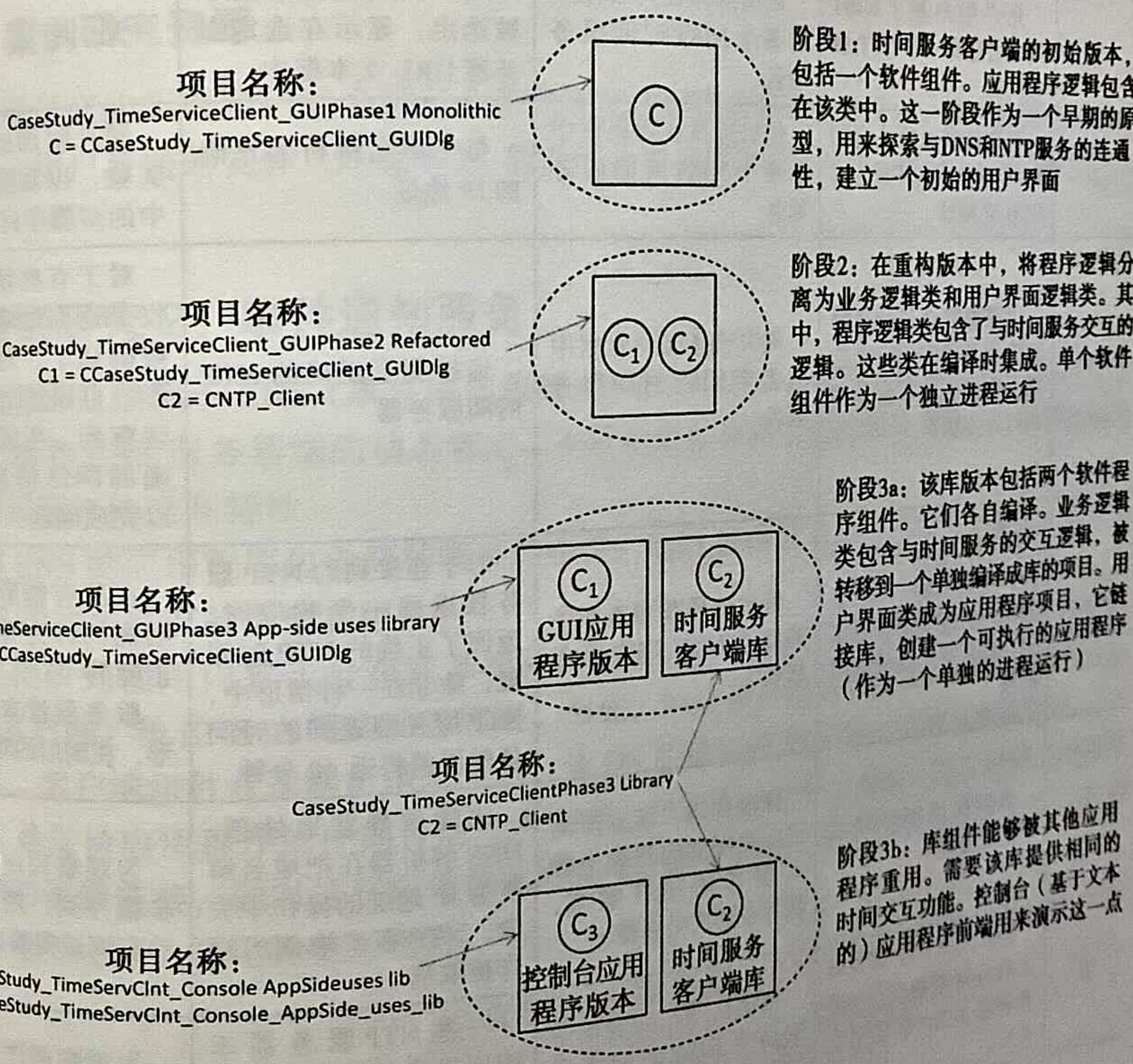


图 7-15 实现线路图

7.3.8 测试

测试是检查需求和设计正确性的一个后续过程，确保设计真正满足了需求，实现真正反映了设计。在 NTP 客户端应用程序的案例中，我们决定使用库的形式实现核心功能。在这个特殊的应用案例中，最重要的行为特性是：正确地使用了 NTP 协议，从 NTP 服务返回了正确结果，这些结果也能从程序代码中得到正确解释。可能产生多种不同的错误，其中一个能感知到的特别应避免的问题是，尽管有可能 NTP 服务本身发生了错误，或者因为与服务的通信导致了错误，都不应该导致应用程序的崩溃或死机。采取的方法是构建一个应用程序，用作测试平台，使得 NTP 协议的交互正确，随后在第二阶段重构、抽取出程序库。用

488

户界面不是第一阶段最需要关注的，不过，因为用户界面的表现非常符合目标，也就因此没有在第二阶段和第三阶段中修改它。

正式的测试计划用作一个最终的签收，认为应用程序满足了其功能、行为和正确性的需求。但正如上所述，由于其持续特性，这些也不能反映实际应该测试的全部范围。表7-1提供了测试计划和测试结果。

表7-1 时间服务客户端应用程序的测试计划和结果

测试编号	测试的特性或行为	测试条件（测试条件和报告结果与应用程序GUI版本有关）	结果	偏差或其他评价
1	允许用户基于NIST的域名选择一个NTP服务实例	依次选择在列表框中的各个NIST时间服务域名	用户高亮的域名选项被选定，显示在选定服务器URL文本框中	无
2	可靠静默地把选定的NIST服务域名解析为相应的IP地址	依次选择在列表框中的各个NIST时间服务域名	每个域名解析为正确的IP地址	展开附加的动态调试实验，以观测程序代码中的步骤序列
3	可靠静默地与合适的NTP服务器实例建立连接；用户应该不会觉察到行为的细节	依次选择在列表框中的各个NIST时间服务域名	连接到了正确的NTP时间服务器	对于有些情形中，接收了返回的响应，这些被确认了。对于其他情形，可以通过动态调试观察到，生成了格式正确的网络信息并发送，以完成确认
4	从NTP时间服务获得NTP时间戳。无论连接哪一个NTP服务器实例，都获得了正确的时间戳	依次选择在列表框中的各个NIST时间服务域名	对于连接到的NTP服务器返回一个响应时，返回了正确的NTP时间戳；除非在一种情形中，NTP服务器返回的时间值的误差将近40分钟	在所有情形下，客户端侧的行为看上去都是正确的 服务器没有响应的情形，在随后的测试中考察
5	静默处理NIST域名不正确的情形	修改代码，以方便测试。在提供的列表中置入虚构的NIST域名，然后在测试中选择	故障被静默地处理。DNS解析器在把域名解析为IP地址的操作中失败，NTP客户端侧的操作被取消	为观察程序代码中的步骤序列，展开额外的动态调试实验
6	静默处理NTP服务器未响应请求的情形（或者请求消息/响应消息丢失）	依次选择在列表框中的各个NIST时间服务域名	一些NTP服务器未响应发来的请求。接收方法按预期方式超时，NTP客户端的操作取消	为观察程序代码中的步骤序列，展开额外的动态调试实验
7	UI外观。用户界面必须清晰、整洁	从静态设计角度和使用实时数据角度，观察用户界面	证实UI有可接受的外观	在一定程度上，这是一个主观结果，因为在这种情形下，测试人员和设计师是同一个人
8	UI功能性。用户界面控件必须功能正常，必须能正确地显示数据值	用真实数据在使用中观察用户界面	控件功能正确，数据值显示正确	通过对比本地计算机时钟和外部卫星控制的时钟，确认NTP时间戳数据的正确性

7.3.9 用例的透明性

对用户的透明性。NTP客户端侧的库，处理与NTP服务的连接和通信。这样，对

用户隐藏了通信特性，而与前端用户界面无关。根据设计情况，用户界面有可能隐藏或暴露不同层面的细节，例如，可能要求用户通过其域名和 / 或 IP 地址来选择一个名称服务的实例。这个库也可能嵌入应用程序中，从而能够从用户角度悄悄处理 NTP 连接，从而没有必要让用户感觉到使用了 NTP 服务。该库提供了位置透明性、分布透明性和访问透明性(时间值是一种资源，通过 NTP 服务获得，而与使用了哪个服务实例或其底层平台无关)。

开发者透明性。NTP 服务的内部时钟的同步对开发者是隐藏的。应用程序只需解析其中一个已发布的 NIST 服务域名，以获得服务器的 IP 地址，然后，向该地址的熟知端口 123 发送一条 NTP 请求消息。所有服务器实例应该提供相同的时间值，从而隐藏了服务的分布特性及其结构和组织，包括时钟层细节和服务器间的同步过程。

7.3.10 案例研究资源

本案例研究的实现经历的全部三个阶段中的完整源代码和工程文件，以及在第三阶段中使用的两个用户界面的变体（库版本），都提供为本书的附带资源的一部分（5 套完整的代码工程）。

489 ~ 491

7.4 案例研究 2：事件通知服务

本案例研究聚焦于 ENS（事件通知服务）的设计与开发。此外，还示范了使用 DS 支持 ENS 客户端和 ENS 服务器端的动态绑定。不同的组件使用不同的语言开发，以示例异构分布式应用程序中的交互特性。

使用 ENS 的应用程序包含两种类型的组件：通过 ENS 发布事件的发布者组件，以及使用从 ENS 接收事件值的消费者组件。

本案例研究使用内置于分布式系统版 Workbench 的 DS，以便于实现在应用程序组件（ENS 的客户端）和 ENS 之间的动态绑定。为此，ENS 服务端自动向 DS 注册其名称和地址详细信息，客户端组件发送解析请求消息，从 DS 获得 ENS 服务器的 IP 地址和端口号。

ENS 是支持应用程序的中间件的一种形式，以发布 - 订阅为基础。ENS 提供应用程序组件之间的逻辑连接，同时这些组件彼此之间保持完全解耦。图 7-16 提供了发生在 ENS 和应用程序组件之间的简化的交互视图。

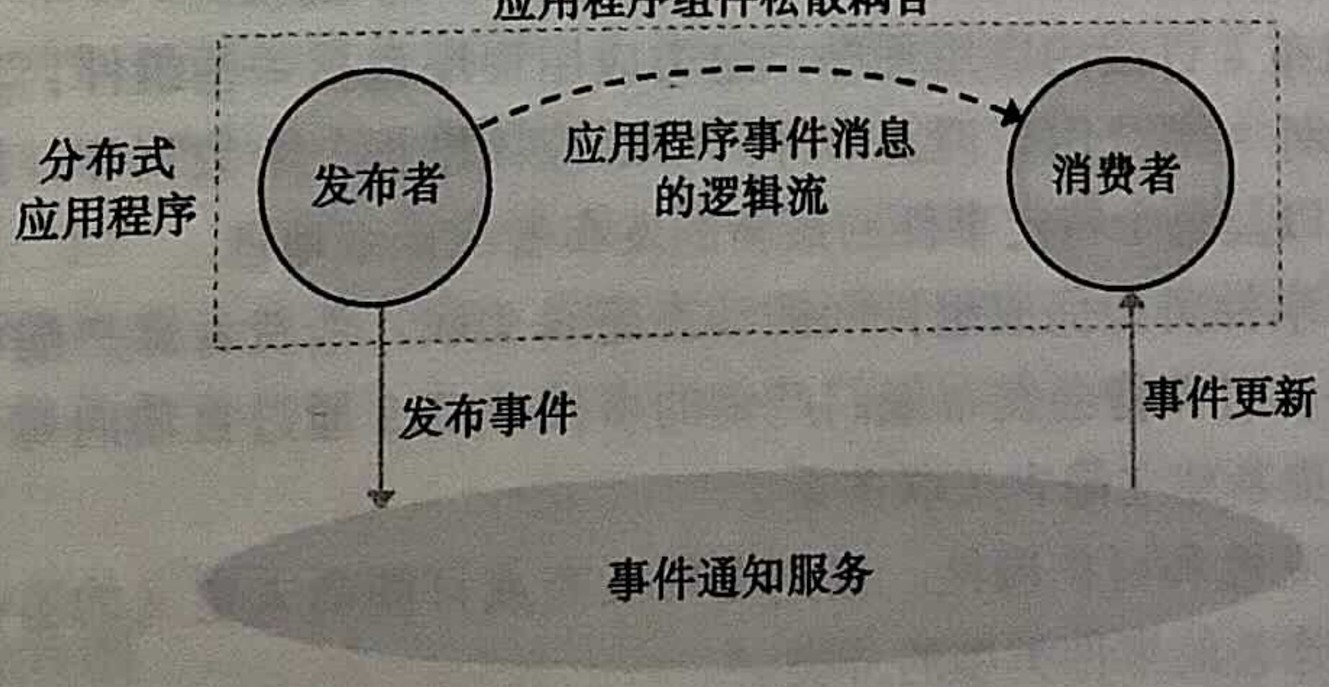


图 7-16 事件通知服务角色的高层说明

图 7-16 展示了 ENS 如何用作应用程序组件之间的中间件。组件发布的事件会自动传递给那些已经注册并对此事件类型感兴趣的组件。

7.4.1 学习目标

本案例研究涵盖了广泛的主题，包括以下内容：

- ENS 实例。
- 发布 - 订阅应用程序实例。
- 松耦合组件和运行时配置的一个实例。
- 使用 DS 实现组件间的动态绑定。

492

- 在同一应用程序中联合使用 TCP 和 UDP 传输层协议的例子。
- 在广播模式下使用 UDP 实现服务定位的例子。
- TCP 连接和一个服务器组件维护一组连接的客户端的例子。
- 使用不同语言开发的异构组件间的互操作能力。
- 面向特定语言的数据表示，序列化和反序列化为与实现无关的字节数组形式。

值得注意的是，应用程序组件（事件发布者和事件消费者）分别用 3 种不同的语言（C++、C # 和 Java）实现，因而，有 6 种不同的示例客户端组件类型。它们能够连接到 ENS，并通过事件发布和消费进行交互。

以不同语言实现的组件有不同的用户界面，其“外观和感受”不同，但执行了相同的底层行为。特别是，它们都使用相同的单一的接口，与 ENS 进行交互。这意味着，通过 ENS，无论组件用哪种语言编写，它们都能够互相操作，例如，一个 C++ 发布者组件能够创建事件，之后被一个用 Java 编写的消费者组件消费（6 种类型组件的任何组合都能工作）。为达到此目标，所有的消息都被序列化成特定的 ENS PDU，它包括 4 个固定长度的字段，并作为一扁平字节数组（一个连续的字节块）发送。

消息到达 ENS 时，它们都有相同的标准格式，这样，就能够从消息中正确地提取出消息内容，并构建适当的应答，而不需要知道发送者的具体实现方式。

7.4.2 需求分析

本节指明了 ENS 的功能性和非功能性需求：

- 自动定位和连接。ENS 服务器应该向 DS 自动注册，以便其他组件查询 DS，按需获得 ENS 服务器的地址。当客户端组件启动时，它应该通过查询 DS，自动发现 ENS 的详细地址。然后，客户端会自动与 ENS 建立 TCP 连接。
- 支持基于发布 - 订阅的应用程序。这类应用程序包括一些组件，其中一部分用于发布事件，另外一部分用于消费事件。ENS 应该维护一个数据库，用于存储事件类型和它的值，以及每个特定事件的最新的发布者的详细地址。
- 仅基于命名事件类型促成组件间的动态逻辑关联。消费者客户端将会订阅感兴趣的事件。发布者组件将会发布他们产生的事件。ENS 通过自动向每个客户端转发其订阅的事件的更新建立隐含关联关系。
- 应用程序独立性和可扩展性。遇到需要发布或订阅尚未定义的事件类型时，ENS 服务器必须创建新的事件类型的分类。
- 低延迟。ENS 服务器在收到订阅请求时，必须立即将事件的当前值（如果知道）传送给订阅者。也就是说，它发送的是即刻可用的缓存值，它不能联系发布者请求更新，这将加大延迟，并提高整体的通信强度。随后，基于状态改变，事件更新传播到这一组注册的订阅者组件。

493

是几种类型事件的发布者和其他事件的消费者时，或者，当组件消费了一个事件，修改底层数据值，然后再发布相同事件的新版本时，设计会更加复杂。ENS 是无状态的，也就是说，它的设计假定是，发布活动和订阅活动是独立的、异步的和无序的。一个特定的事件类型可能会有多个订阅者，这种情况没有问题。然而，当一个特定的消费者订阅了多个事件类型时，更新的相对顺序可能在一些应用程序中会出现问题（并且，应该在应用程序的逻辑中解决）。如果一个特定的事件类型有多个发布者，可能会需要设定面向应用程序的限定条件，例如，使用附加的命名信息创建事件类型的变体，以示区分，从而避免产生对特定发布者组件的依赖性。例如，一个发布者可能发布一个事件类型 temperature_01，表示精度为 1° 的温度值；而另外一个发布者可能发布一个事件类型 temperature_03，表示精度为 3° 的温度值。消费者可能同时订阅两个事件类型，在事件类型 temperature_01 可用的情况下，优先使用它。

7.4.5 组件之间的耦合与绑定

ENS 客户端（无论是应用程序的发布者组件，还是消费者组件）之间的耦合关系是松散的（因为在运行时用 DS 动态查找 ENS 服务器的地址详情）和直接的（因为一旦获得 ENS 服务器的地址详情，客户端组件和 ENS 服务器之间将直接建立一条 TCP 连接）。

应用程序组件自己之间的耦合关系，也就是说，事件发布者和事件消费者之间的耦合关系是松散的（因为 ENS 提供了基于事件类型的逻辑连接，而不是基于地址的连接）和间接的（因为这些组件之间没有直接的连接或消息传递；所有的通信都通过 ENS 服务器代理）。

避免紧密耦合，确保整个系统，包括 ENS 服务和使用 ENS 服务的应用程序，都能够在运行时灵活配置，而且，任何组件都不需要关于其他组件位置的先验知识。这也有助于健壮性，因为能够替换发布者组件，而不影响消费者组件，同样，消费者组件也能够添加或移除，而不影响发布者组件。

应用程序层的耦合关系

逻辑上，应用程序组件只在它们共享的事件关系上产生耦合。每当一个发布者发布一个新的事件类型，或每当消费者预订了一个 ENS 数据库中还没有的新事件类型时，就动态创建一个新的事件类型。这种特性完全是运行时配置的。

因此，应用程序组件对其他组件没有直接依赖，发布者可以发布一个新的事件类型，而不论是否有任何消费者订阅该事件类型，类似地，消费者可以订阅某个事件类型，而不管该事件类型的发布者是否存在。

解除直接依赖对规模可扩展性和功能可扩展性是重要的，因为应用程序的新特性，甚至是新的应用程序，都能够由 ENS 支持，而不必预先知道有哪些事件类型。应用程序层的完全解耦允许发布和订阅操作可以按任何顺序异步进行。例如，订阅尚未发布的事件类型也不是一个错误。图 7-18 示例了松散耦合的实现。

图 7-18 展示了 ENS 如何解耦应用程序组件，使得它们没有直接依赖，也不用直接通信。这允许系统扩大规模，而不用重新设计组件。例如，可以为特定的事件类型添加新的消费者，而不必修改产生该事件的发布者组件。解除直接依赖关系也提升了错误容忍方面的能力：消费者可以发生错误，而不需要发布者知道。发布者发生错误时，不会导致消费者的崩溃（尽管这些事件将不再由特定的失效的发布者发布，可能启动了另一个发布者，或者消费

者将等待，但这时它仍有能力响应其已经订阅的其他事件类型）。

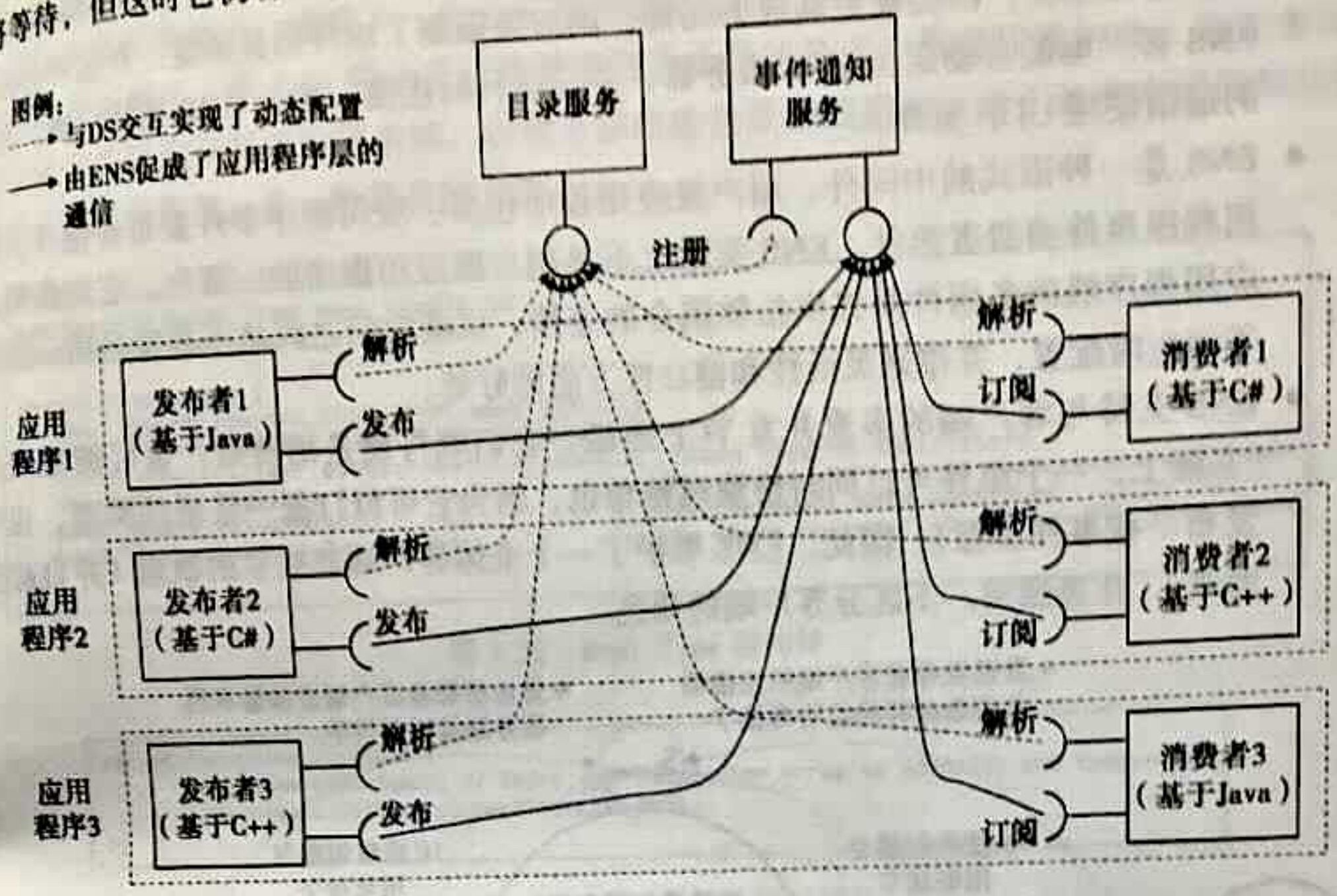


图 7-18　通过 ENS 促成了应用程序组件之间的松散耦合

图 7-18 还展示了两个层次上的关注点分离。图中，展示在 ENS 下面的所有组件，从 ENS 服务的角度来看，都是客户端。然而，从特定应用程序的角度看，左侧的组件实质上是服务器端，而右侧的组件实质上是客户端。

7.4.6　设计的通信特性

图 7-19 展示了在 ENS 和 DS 中使用的端口的分配情况。

```
#define From_DIRECTORY_SERVICE_PORT  (u_int) 8002   // UDP, Defined by Directory Service
#define To_DIRECTORY_SERVICE_PORT    (u_int) 8008   // UDP, Defined by Directory Service
#define ENS_APPLICATION_PORT         (u_int) 8003   // TCP, ENS_Server Binds to this port
```

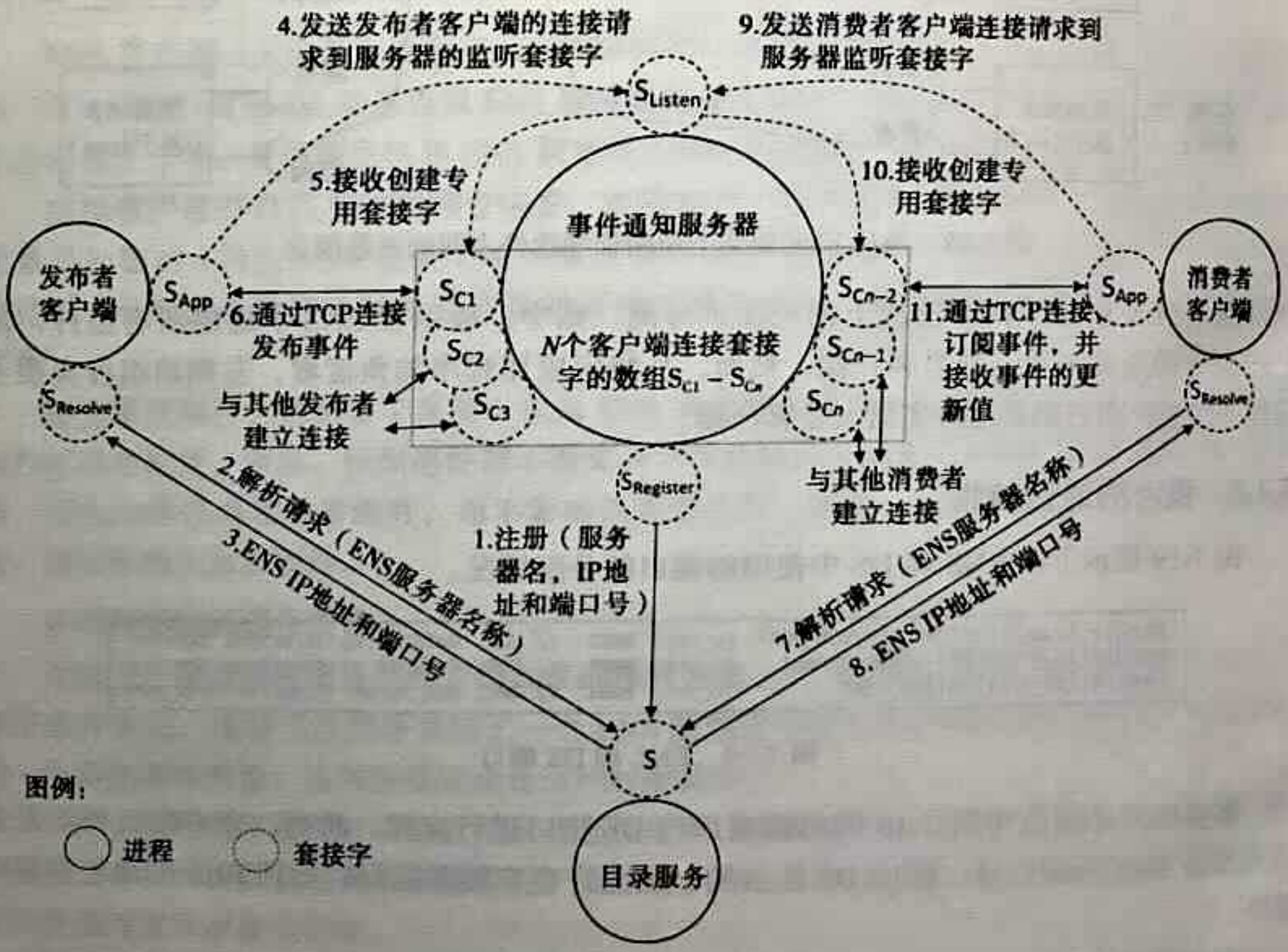
图 7-19　ENS 和 DS 端口

所有组件必须使用图 7-19 所示的端口与 DS 接口进行交互。然而，客户端组件不需要预先知道 ENS 的端口号。使用 DS 的目的，是允许客户端在运行时找到 ENS 的地址和端口细节。

ENS 综合使用了 UDP 和 TCP 传输层通信。UDP 用于与 DS 通信。UDP 广播用于向 DS 发送注册消息（对于 ENS 服务器），也用于向 DS 发送解析请求消息（对于 ENS 客户端，目的是获得 ENS 服务器的地址）。DS 用一条发送给特定客户端组件的定向 UDP 消息，来响应解析请求。TCP 用于 ENS 服务器和其客户端之间的通信（事件的发布者和消费者）。每一个客户端（发布者和消费者）建立一条与 ENS 服务器的专用 TCP 连接，并借此连接传递 ENS 应用程序消息。有 4 种 ENS 消息类型，将稍后讨论。图 7-20 中，以套接字层面的通信视角的形式描绘出了 ENS 的连接关系。

图 7-20 提供了一个发生在不同组件之间的套接字层面上的通信视图。该图说明了几个要点：

- ENS 服务利用了另一服务（DS）以便于连接。这是一个有用的例子：使用公共服务作为基础模块，以实现更高级的功能，同时还限制了组件的复杂度。在这种情况下，ENS 客户端能自动定位 ENS 服务器，并在运行时连接。鉴于 UDP 支持广播，与 DS 的通信使用 UDP 实现。
- ENS 是一种形式的中间件。用户级应用程序包括：应用程序事件发布者组件，和应用程序事件消费者组件。ENS 实际上不是用户级应用程序的一部分，它的作用是在应用程序级的各组件间提供松散耦合的连接，以便它们之间从不直接通信。这有利于运行时配置，并带来灵活性和健壮性方面的好处。
- ENS 支持与客户端的多重并发 TCP 连接。它们既可能是消费者，也可能是发布者（实际上，一个组件可以同时扮演两种角色，因为它可以订阅一些事件类型，也可以发布一些事件类型）。因此，ENS 维护了一个全体客户端套接字的数组（并自此连接起来），在连接层，不区分客户端的角色。



497

图 7-20　套接字层面的 ENS 通信视图

图 7-21 展示出了 ENS 使用的 TCP 连接结构体。维护了一个该结构体的数组。数组条目中的 bInUse 标记项设置为真，表示它目前连接到了客户端组件。

图 7-22 展示了 ENS 用于保存事件类型细节的结构体。ENS 维护了一个这些结构体的数组。

对于每个事件类型，可能有很多订阅者，而且无法预测不同事件类型的流行程度。如果实际上只有少量订阅者，使用一个大数组来保存订阅者和每个事件类型的

```
struct Connection {
    SOCKET m_iConnectSock;
    SOCKADDR_IN m_ConnectSockAddr;
    bool bInUse;
};
```

图 7-21　ENS TCPconnection 结构体

细节，将效率较低，而且，不管数组设多大，都不能保证该数组在所有情况下都足够大。因此，在每个事件类型上，设置了一个订阅者的链表（如在图 7-22 中所见），它可以根据需要，动态地增大或缩小。图 7-23 展示了订阅者的链表条目的格式。值得注意的是，链表条目仅包含了单个关联到订阅者的数据域，也就是指向连接数组的索引。鉴于这种方式的简洁性，并避免了数据的重复，是一种高效的方法。

```
#define MAX_EVENT_TYPE_NAME_LENGTH 50
#define MAX_EVENT_VALUE_LENGTH 50

struct Event_Type {
   char cEventType[MAX_EVENT_TYPE_NAME_LENGTH];
   char cEventValue[MAX_EVENT_VALUE_LENGTH];
   SubscriberListItem* pSubscriberList_Head;// Linked list of subscribers
   SOCKADDR_IN PublisherAddress;            // IP address and Port of most recent publisher
   bool bInUse_ValueValid;                  // Has the event been published
   bool bInUse_SubscriptionExists;          // Are there consumer(s) waiting for this event
};
```

图 7-22　Eent_Type 结构体


```
struct SubscriberListItem {
       int iConnectionIndex; // Index into connection array to identify the subscriber
       SubscriberListItem* pSubscriberList_Next;
};
```

图 7-23　订阅者链表的表项结构体

1. ENS 消息类型

ENS 应用程序接口包含 4 种消息类型。在所有组件中，它们以枚举类型的形式定义，与组件的角色和组件的开发语言无关（见图 7-24）。这是获得全局互操作能力的一个关键因素。

```
enum EventNotificationService_Message_Type {
   SubscribeToEvent,      /* value 0, sent from Consumer component to ENS */
   UnSubscribeToEvent,    /* value 1, sent from Consumer component to ENS */
   PublishEvent,          /* value 2, sent from Publisher component to ENS */
   EventUpdate            /* value 3, sent from ENS to Consumer component */
};
```

图 7-24　ENS 消息类型代码的枚举结构

各种 ENS 消息类型共享一个 PDU 格式，如图 7-25 所示。这简化了设计和开发。在某些消息类型的表示中，有几个字段没有用到，稍显低效（例如，cEventValue 字段在订阅消息中无效）。不过，这没有考虑其在保证更复杂的设计方案中的重要性。从某种意义上讲，消息的序列化和反序列化都将依赖于消息类型。

```
struct EventNotificationService_Message_PDU {
   byte iMessageType;
   char cEventType[MAX_EVENT_TYPE_NAME_LENGTH];
   char cEventValue[MAX_EVENT_VALUE_LENGTH];   // Not valid in Subscribe messages
   SOCKADDR_IN SendersAddress;
};
```

图 7-25　ENS PDU

图 7-25 展示了 ENS PDU（用 C++ 语言定义的描述）。尽管该结构体无法在实现示例应

用程序中的3种不同的语言中以完全相同的方式定义，但是PDU的格式在所有组件的实现中不变。当该结构体按照如图7-26所示的方式序列化时，得到了其等价的扁平字节数组格式。这形成了117字节的线性缓冲区，是该消息在TCP段传输的真正格式，也是其他语言中，序列化其内部表示时必须遵照的格式。

字节数组中的索引位置	字段长度	字段名	数据类型
0字节	1	iMessageType	取值范围为0～255的无符号整数
1～50字节	50	cEventType	8位ASCII字符值，以ASCIIZ字符串格式（即以字符“\0”终止）
51～100字节	50	cEventValue	8位ASCII字符值，以ASCIIZ字符串格式（即以字符“\0”终止）
101～116字节	16	SendersAddress	取值范围为0～255的无符号整数。每个IP地址部分的编码为单独的8位值。端口号编码为一个16位数值，占2个相邻的字节

图7-26　ENS PDU的扁平字节数组表示

2. ENS通信语义

与ENS的通信运行在一个单向通信的基础上，与4种消息类型的其中之一相关。在应用程序协议层面上，没有必要设置确认机制，因为使用了可靠的TCP传输层协议，当消息丢
499 失或损坏时，TCP协议会自动重传。从概念上看，订阅可能看成是以请求－应答为基础的运行（因为它的应答是零个或多个事件更新消息），但是，由于这些都是异步的，而且可能从不发生，所以，从通信语义的视角看，将订阅消息和更新消息看作两个独立的活动会更加准确。

3. DS消息类型

DS有一个简单的接口，包括两种请求消息，它们均以一个8位的ASCII字符串值编码，按照ASCIIZ格式（即以“\0”字符结尾）和单个PDU应答格式，见图7-27和图7-28。

REGISTER:<*Service name*>:<*Port*>
例如，ENS发送“REGISTER:Event_NS:8003”

RESOLVE:<*Service name*>
例如，一个ENS客户端发送“RESOLVE:Event_NS”

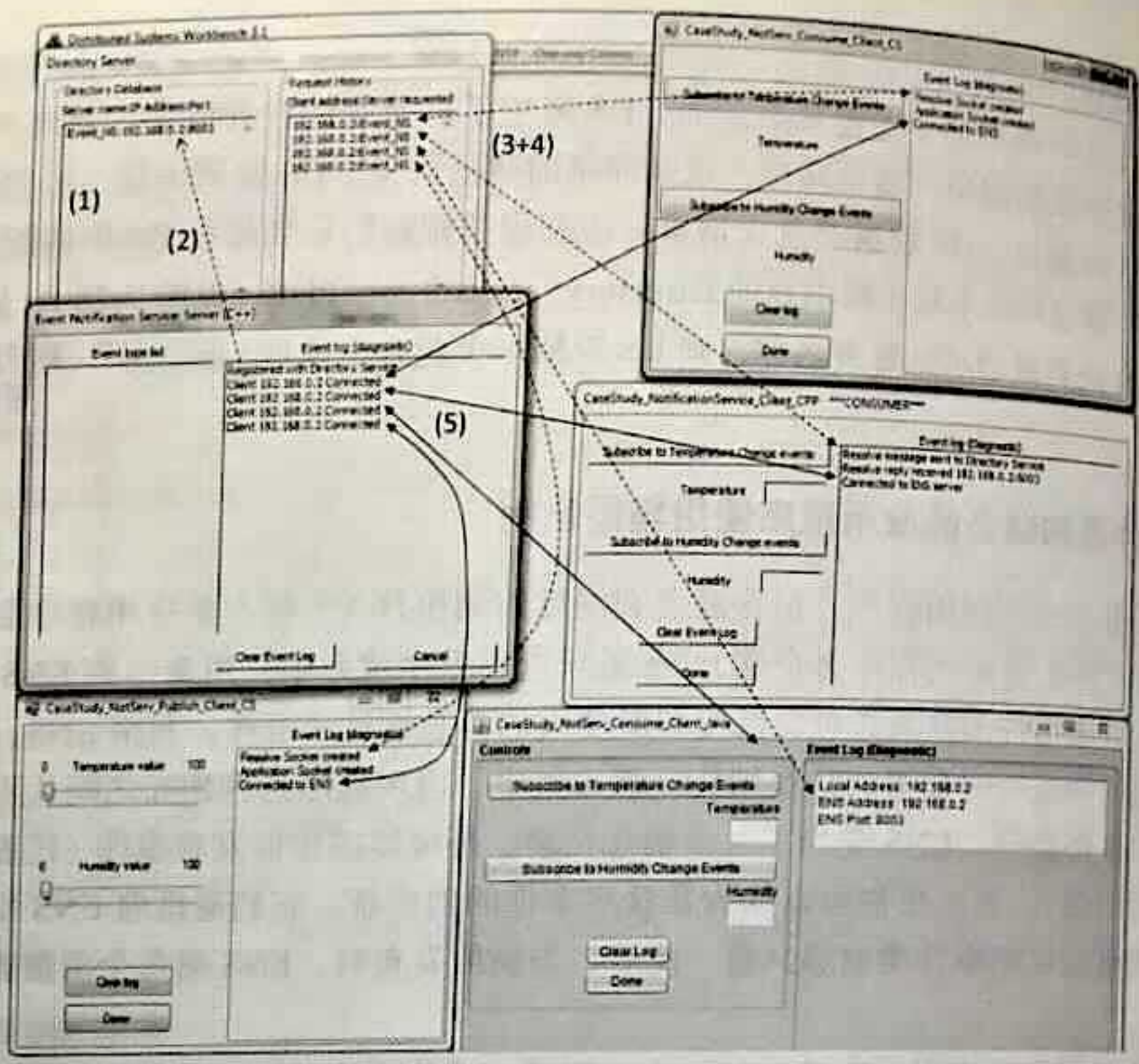
图7-27　以ENS用法举例的DS请求消息格式

```
struct DirectoryServiceReply {
   unsigned char a1;
   unsigned char a2;
   unsigned char a3;
   unsigned char a4;
   unsigned int iPort;
   };
```

图7-28　DS应答消息PDU

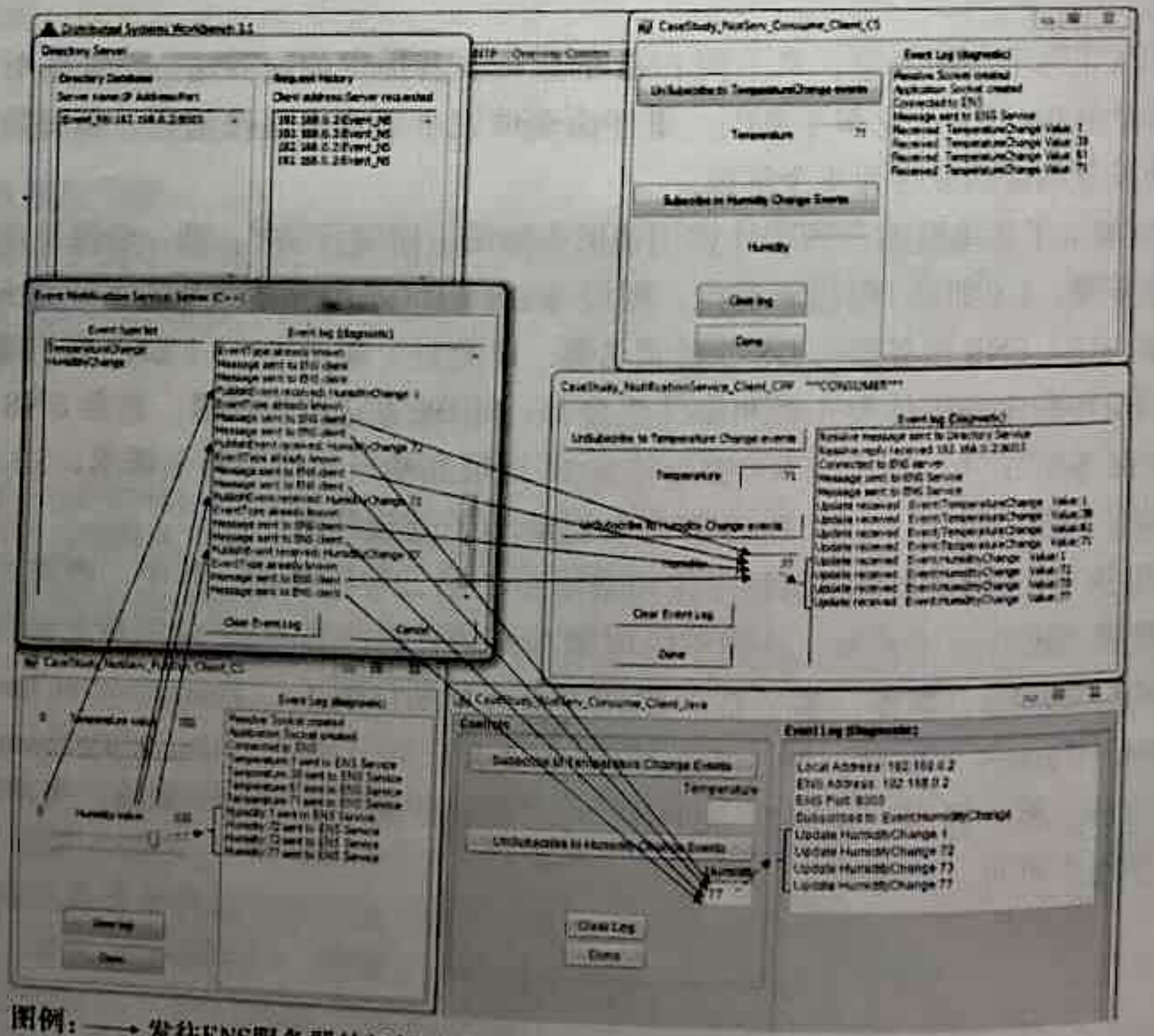
图7-27展示了DS请求消息的两种类型。注意到，对于注册请求消息，DS自动确定发送者的IP地址（通过recvform socket API调用，从UDP数据报头中提取该信息）。因此，没有必要在注册请求消息中显式地为地址编码，这种省略的表示，在处理时间和通信带宽使用率方面，带来一定的提升。而端口必须显式声明，因为我们不能假设注册服务使用了与连接DS相同的端口（用来接收应用程序消息）。

500 图7-28展示了DS应答消息的PDU，其中包含IP地址和被请求服务的端口号。



图例：◆·····▸与目录服务的通信　　◆——▸与ENS服务器的通信

图 7-29　阶段 1：目录服务促成了自动绑定和连接



图例：——▸发往ENS服务器的订阅请求　·····▸事件日志更新，以反馈订阅消息正在发送

图 7-30　阶段 2：消费者客户端订阅他们感兴趣的事件类型

值得注意的是，本节示例的特定应用程序的场景序列中，3 个消费者都在这些事件发布前订阅了事件类型。发布行为和订阅行为是完全异步的，并且，ENS 不强制或期望任何特定的顺序。例如，也许，一个消费者比较晚才订阅了某种事件类型，已经发布了该事件类型的一些值，在这种情况下，一旦接到订阅消息，ENS 服务器就将其存储的相关事件类型的最新值发送回客户端。

图 7-31 展示了活动的第三阶段。基于 C# 的发布者发布了一系列 4 个 TemperatureChange 事件。每个事件都以事件更新消息的形式发送给相应的订阅者。需要注意的是，只有已经订阅了该特定的事件类型的消费者客户端才能接收到更新。本案例中，指的是基于 C # 的消费者和基于 C ++ 的消费者。

502

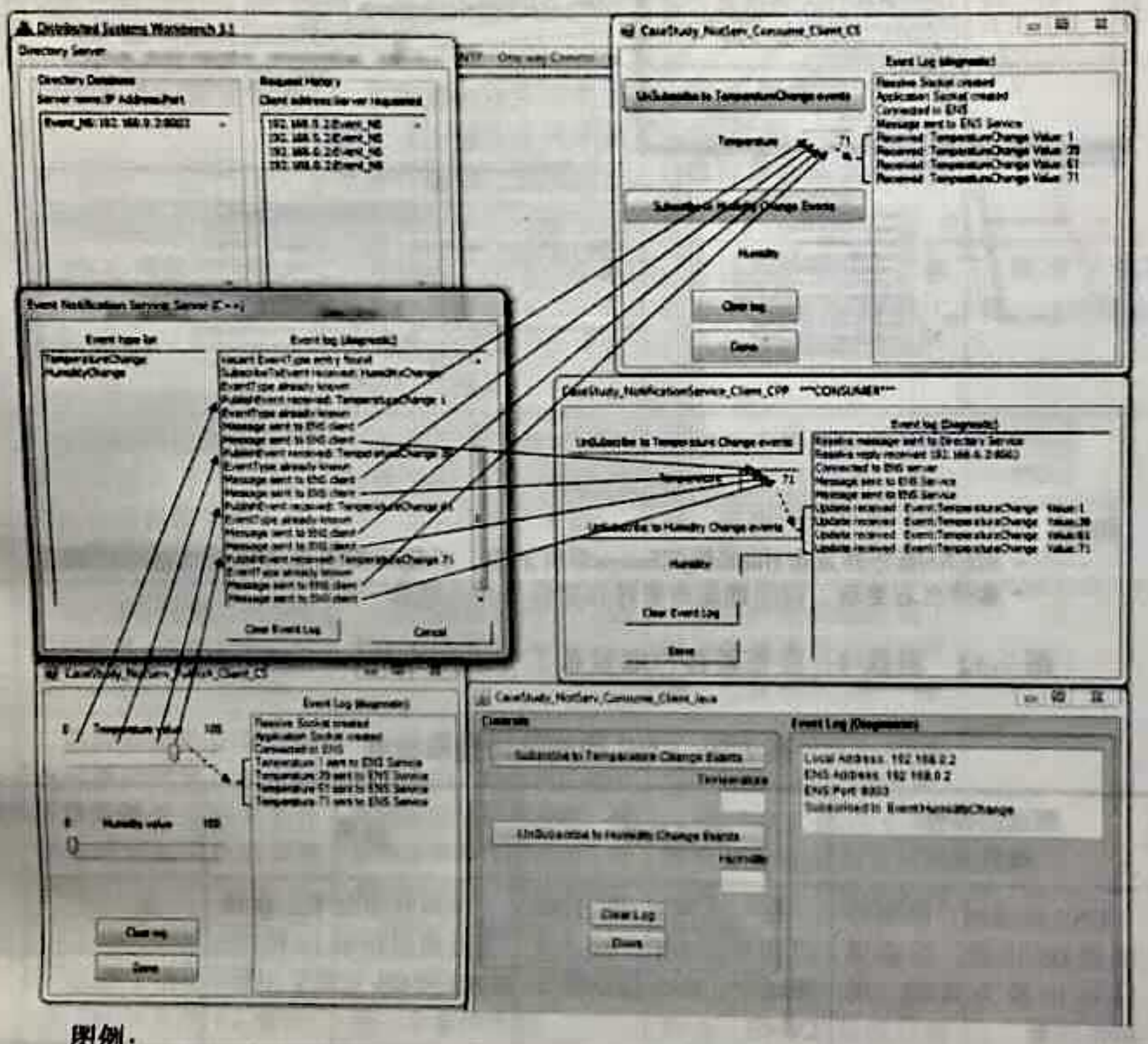
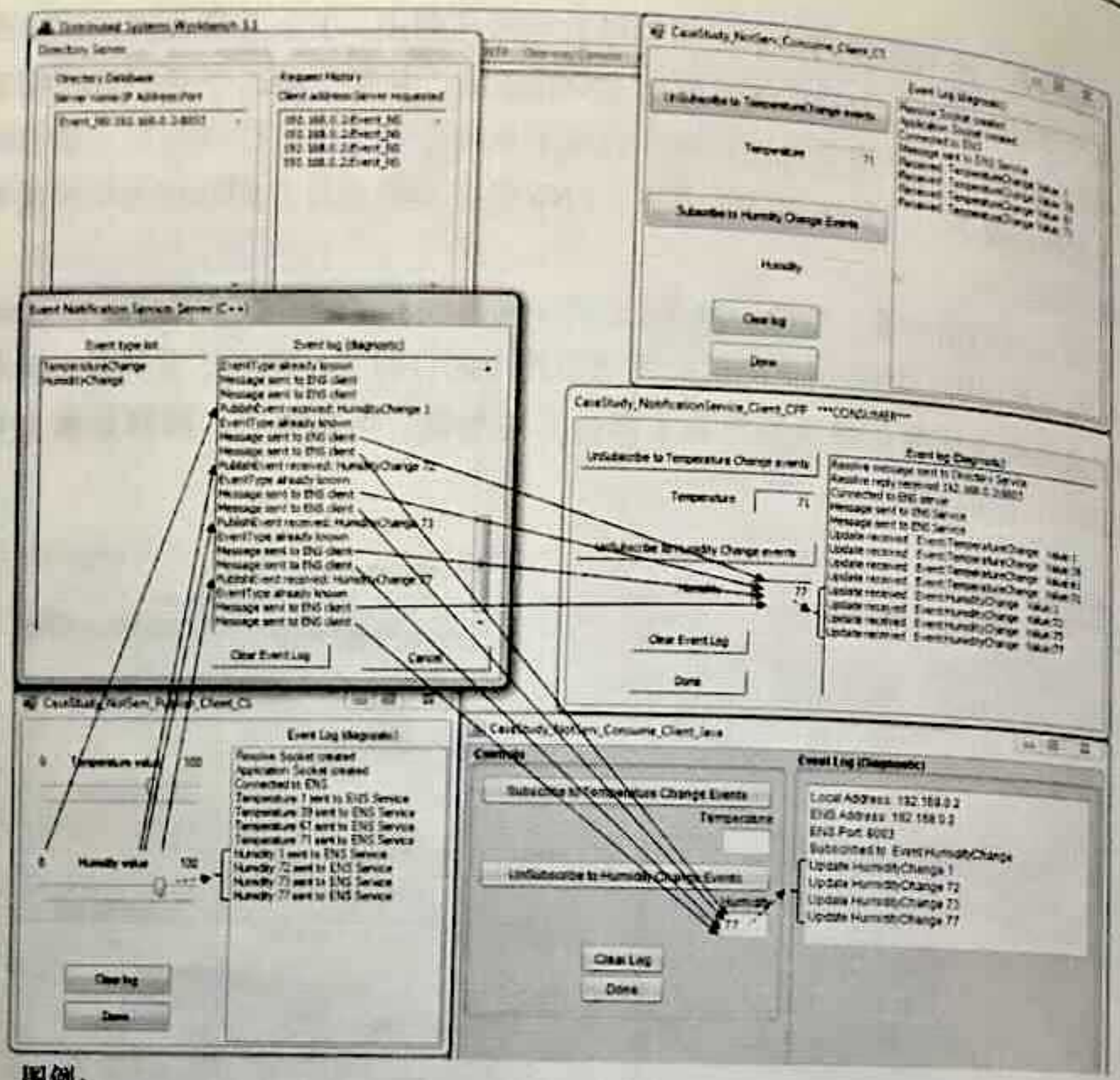


图 7-31 阶段 3：发布者客户端发布了一系列 TemperatureChange 事件

图 7-32 展示了活动的第四个阶段。基于 C # 的发布者发布了一系列的 4 个 HumidityChange 的事件。每个事件都以事件更新消息的形式发送给相应的订阅者。需要注意的是，只有已订阅了 HumidityChange 事件类型的消费者客户端才能接收到更新。在本案例中，它们指的是基于 C++ 的消费者和基于 Java 的消费者。

7.4.8 测试

测试计划和测试结果在表 7-2 中展示。



图例：

——→ 向ENS服务器发布HumidityChange事件消息，并随后发送事件更新消息给订阅者

·····→ 事件日志更新，以反馈发布事件和更新事件的活动

图 7-32　阶段 4：发布者客户端发布了一系列的 HumidityChange 事件

表 7-2　ENS 的测试计划和结果

测试编号	测试的特性或行为	测试条件	结果	偏差或其他评价
1	ENS 启动时，必须自动向 DS 注册，且必须以每 10 秒为基础，周期性注册	在一次测试中，ENS 启动早于 DS；另一次测试中，ENS 启动晚于 DS	在两种情况下，在两个进程运行的 10 秒内，ENS 向 DS 完成了注册	无
2	ENS 的客户端（它们是应用程序的发布者和消费者组件），必须能够从 DS 获得 ENS 的地址和端口详情	在 ENS 已向 DS 注册和尚未注册两种情形下，客户端组件向 DS 发送解析消息	如果 ENS 已在 DS 中注册，客户端组件获取了 ENS 的地址和端口详情	DS 保存服务器地址细节，直到被重写。因此，如果 ENS 注册后意外终止，DS 将会继续提供 ENS 地址详情。如果 ENS 继续缺席，这成为一个问题。然而，如果 ESN 重新启动（无论是否在同一台计算机上），发送一个新的注册消息，则问题会自动解决
3	ENS 客户端应该使用从 DS 获得的地址详情，与 ENS 建立 TCP 连接	测试发布者组件和消费者组件，以确定它们是否自动连接到 ENS	客户端组件的确与 ENS 建立了 TCP 连接	

（续）

测试编号	测试的特性或行为	测试条件	结果	偏差或其他评价
4	ENS服务器必须能接受多个TCP连接，直到设计支持的上限	对该代码进行了修改，以方便测试。将连接上限（在常量MAX_CONNECTIONS中定义）手工降低为5，然后用1来测试接受/拒绝行为	接受的连接上升到极限，其细节以连接数组条目的形式保存（m_ConnectionArray）	为了观察程序代码内部的步骤序列，添加了动态调试实验
5	应用程序组件必须完全解耦，并有独立的行为和生命周期	以事件联系相关的发行者组件和消费者组件，按不同的顺序启动和停止	组件的行为，包括突然终止，对其他组件未有不良影响	从未有消息直接由一个应用程序组件发送到另一个
6	ENS消息必须完全异步	发布者组件和消费者组件以不同的序列和顺序执行发布、订阅和退订操作	在全部情况下，如果有订阅，则ENS推送事件更新消息给消费者，否则不推送	
7	ENS必须接受已发布或订阅的新的事件类型（之前未知）	发布者产生了一个新的事件类型E1 消费者订阅了一个新的事件类型E2	在这两种情况下，新的事件类型添加到了ENS的事件类型数组中	修改了组件的源代码，以方便这些测试
8	应用程序组件必须能够发布多种事件类型，并且必须能够为现有的事件类型发布新值	发布者用于生成多个事件类型，也为每个事件类型生成多个更新	ENS正确地接收到全部的“事件类型－值”对，并相应地更新事件类型数组	
9	组件必须能够订阅多种事件类型，并能接收每个事件类型的多个更新值	一个消费者用于订阅多个事件类型	该组件的确接收到了每个事件类型的更新序列，与发布序列相一致	
10	消费者组件必须能够退订事件类型，且ENS应该不再推送相关事件类型的更新消息给该消费者	一个消费者订阅了两个事件类型。两者都正在由其他组件活跃地发布。接收到各种事件类型的几个更新消息后，消费者退订了其中一个事件类型	消费者接收到其订阅的事件类型的部分事件更新。 退订语义操作正确，消费者继续接收其订阅的剩下的一个事件类型的更新	
11	必须支持多组件，组合成任何角色（a部分：发布者组件）	多个发布者用于发布同一个事件类型	无论发布者的标识是什么，ENS总是推出给定事件类型的最新值	
12	必须支持多组件，组合成任何角色（b部分：发布者组件）	多个消费者用于订阅同一个事件类型	ENS向所有已经订阅的该事件类型的消费者推送事件更新	
13	ENS必须独立于应用程序	在一些组件中改变事件类型字符串，测定是否可以同时支持多个不同的“发布者－消费者”应用对	ENS独立地处理每个事件类型，向正确的消费者推送正确的事件更新，无需重新配置或向多个应用程序提供知识	修改了一些组件的源代码，以方便测试
14	无论ENS应用的实现语言是什么，应用程序都必须支持	消费者组件和发布者组件由C++、C#和Java开发（总共有6种例子组件）	测试了各种发布者组件和消费者组件的组合。所有组合运行正常	这表明ENS方法提供了互操作性支持，并强调了ENS是一种形式的中间件的概念

7.4.9 事件通知服务的透明性

因为应用程序发送数据时，采用了平台无关且语言无关的序列化字节流格式，自然提供了访问透明性。所有事件类型名称和事件值，均以简单的 8 位 ASCIIZ 字符串格式表示，从而避免了编码异常。如果数据格式为整数和浮点数，在不同的系统中，使用了不同的编码方式和精度，往往会引起编码异常。字符串的方法还意味着，任何事件类型的名称和值都能够在应用程序全然不知的情况下，为 ENS 编码和管理。

在客户端需要连接时，使用 DS 解析 ENS 的名字到其地址，这事实上提供了位置透明性。

ENS 服务器以每 10 秒的周期自动向 DS 注册，从而实现了迁移透明性。这意味着，它可以在计算机之间移动，所以任何新启动的客户端组件将收到新的位置详情，只要它们启动后向 DS 发送了解析请求。

503 ~ 504

实现透明性。结合如上所述的简单数据表示（参见访问透明性），使用了标准的 TCP 通信协议。这意味着，ENS 支持客户端运行在任何平台，并支持用任何语言开发。然而，如果应用程序代码中执行消息的序列化和反序列化动作，就的确需要格外小心（参见包含在支持材料中，在 C++、C# 和 Java 等相关例子中使用的不同技术）。

由 ENS 促成的应用程序组件之间的松散耦合有助于实现故障透明性，因为各组件之间没有直接的依赖关系。一个组件崩溃，不会导致其他组件崩溃，从而实现了故障容忍。这里的透明程度，实际上就是应用程序依赖。例如，如果消费者组件的崩溃不会影响到发布者组件的行为，那么，在系统的这些地方，就不再需要消费者角色。类似地，如果发布者出现故障，它产生的事件将不再发布。这不会导致消费者崩溃，但这些消费者有可能会无限期地等待缺失了的事件。事件更新的异步特性，意味着消费者仍然有可能对已订阅的其他事件产生响应（这取决于应用程序事件的专门解释）。

此外，ENS 的运行设计为独立于应用程序，也就是说，它能够支持任何基于发布-订阅的应用程序。这有助于技术的灵活性和技术不会过时。

7.4.10 案例研究资源

ENS 服务器是用 C++ 开发的。环境监测应用程序的开发，使用了 3 个发布者组件变体（用 C++、C# 和 Java 开发）和 3 个消费者组件变体（也用 C++、C# 和 Java 开发）。

对于 ENS 服务器和环境监测系统（ENS 的客户端）的应用程序组件，其所有源代码和工程文件都作为随书提供的支持资源的一部分。分布式系统版 Workbench（它提供了运行 ENS 系统所必需的 DS），也作为支持资源材料的一部分提供。

7.5 分布式应用程序的优秀设计实践

505 ~ 508

本节总结了主要的设计理念。这些理念在本书中已经明确过，并在相关案例研究中讲述过。本节针对一个分布式应用程序的设计，结合讨论各种问题的重要性，指出了必须考虑的关键事项和做出的决策，从而提供了指导原则。

7.5.1 需求分析

争取需求分析正确。投入足够的耐心，确保你理解了需要什么，需要哪些特性，以及它应该如何工作。你也应该考虑可行性，制定初步计划，说明你如何构建和测试应用程序。如

果你在这一阶段无意中犯了错误，这些错误将会在整个设计和构建过程中蔓延。很多系统实际上都失败在了这一阶段，但这种错误直到很后来，才逐渐变得明显。从这一点上看，这浪费了时间和其他资源，且越发难以纠正。

需求总共可分为两类：功能性需求和非功能性需求。两者同样重要，但功能性需求更容易明确，通常其测试也更加简单。非功能性需求可能会被忽略，或者，有时会被忽略，希望在后期添加。然而，通常情况下，往往不可能有效地在后期添加对非功能性需求的支持，因为从其本质上看，非功能性需求不能由特定软件功能提供（例如，可以考虑透明性和规模可扩展性）。相反，它们是优秀设计实践的全方位综合结果，包括功能行为、底层软件体系结构、机制特性等。因此，在项目的每个阶段，把非功能性需求作为头等考虑是非常重要的。

7.5.2 架构方面

在逻辑基础上分离关注点，需要受到重视。这在同一个软件组件中的类层面上适用，也在软件组件层面上适用。这些软件组件分别以独立的进程运行。例如，在“客户－服务器”应用程序中，需要重视客户端与服务器之间的功能划分。再例如，考虑分布式游戏案例，其中，服务器管理游戏逻辑，并保存游戏状态，而客户端提供用户界面。关注点分离带来了很多好处，包括：设计简洁，特别是在接口方面，跨过接口的交互要尽量少；更易于理解代码的结构和行为（并因此更易于测试、验证和编写文档）。特别地，清晰地分离关注点，能够减少进程间通信的强度，从而减少了网络带宽的占用，带来更小的延迟。在清晰的边界上，分离组件的逻辑或行为范围，也能减少产生直接依赖，因为，相比于把一项特定功能分散在若干个组件之间，稍有更紧密的组件集成度。

尽可能避免直接耦合。当一个组件无法运行是因为其他组件发生故障或缺失时，直接耦合就会面临故障传播的风险。直接耦合也会把设计时的连接视图投影到（通常情况下，这不能预见可发生的全部配置情形）运行时。松散耦合更加灵活和健壮，根据实际的可用性和其他上下文特性，它允许组件间建立运行时的连接关系，比如系统工作负载和资源重新配置等，克服了平台级的故障。

组件应使用恰当的规模。粗粒度设计方案中，组件较大，能减少组件的总数目，从而也减少了把应用程序连接在一起时必要的通信数量。然而，如果这种方法做得过了火，将减少分布式带来的优点。比如，这些优点有：能够分离功能、处理资源间分散负荷，便于共享，以及满足非功能性要求（例如，可用性、健壮性和规模可扩展性）等。更大的组件也更复杂，并因此在测试方面面临更大的挑战性。细粒度的系统，有很多小的组件，会更加灵活，单个组件更容易理解和测试。然而，如果系统的粒度过于细碎，则面临的风险是，产生了过度的连接和更高的额外通信开销。因为通信组件的数量在增加，系统的配置复杂度增大，同时，这换来了单个组件的复杂度降低。因此，找到一个平衡点非常重要。其中，对于特定应用来说，按照其功能和连接需求，组件采用了恰当的规模。

使用公共服务提供通用的功能，从而保持应用程序组件尽可能简单。应用程序的功能应主要聚焦于业务逻辑，尽可能不提供额外的服务，如资源定位。使用公共服务，保证整个系统和应用程序中产生标准化的行为。公共服务本身能够以模块化的方式使用，正如在本章的第二个案例研究中，其 ENS 利用了外部的 DS。本来，也可以选择在 ENS 中实现 DS 功能，或使用某种形式的服务宣传广播机制。然而，这将额外增大 ENS 的复杂度，可能会减弱核心功能的开发。在很多系统中，其他应用程序和服务可能也很需要 DS 功能。在这种情况

下，实际上提供 DS，不需要额外的开销（也就是说，它不为 ENS 专用）。

7.5.3 通信方面

通信是所有分布式应用程序的基础，也是产生许多不同问题的根源或影响因素。这些问题发生在性能、延迟、资源限制、错误等相关方面。通常情况下，在仔细分析应用程序需求的基础上，使通信频次最小化，保持消息尽可能短。必须以与语言无关、体系结构无关的格式序列化消息和发送消息，确保在异构系统中的一致性和正确性（见事件通知服务案例研究）。

阻塞 IO 模式的通信效率高，因为进程（或线程）在等待消息到达时，不占用 CPU 资源。然而，如果进程无限等待，则可能导致无响应的进程行为（见第 3 章）。通过多线程设计，能够克服这一问题。这样，当一部分线程等待 IO 时，另一部分线程能够继续响应。作为另外一种选择，通过使用非阻塞 IO 模式套接字，并选择合适的超时机制，单线程组件也能够通过设计来保持其响应能力，而不用多线程等额外复杂度（例如，参见时间服务客户端案例研究中使用的技术）。这种级别的细节设计只能在特定应用需求背景下精心考虑。不过，有必要指明这一特性，因为它能够明显地影响效率。必须避免在一条通信连接的两端都使用阻塞 IO 模式套接字的设计模式，因为这会出现通信死锁的风险（参见第 3 章）。

7.5.4 尽可能重用代码

代码重用有很多好处。最显而易见的是，你不必写很多代码。然而，还有其他好处，也许更重要，说明如下：

- 更好的代码可读性。只要有机会，抽取类和方法的做法，其效果将会减小方法规模，或代码块数量，从而使代码更容易理解。额外的好处是，某些错误或低效因素将更有可能在编写代码时就被发现了。

510

- 更好的代码结构。重构改善了代码结构，同时，避免了重复代码（通过抽取成多个方法），使代码更易于维护，因为未来的修订都仅需要在一处完成；这也防止了在增量更新过程中，不知不觉产生的不一致性。典型情形是，多处代码需要更新，而且这些修订需要关联同步。
- 减少测试工作量。重用已经通过单元测试的代码块，节省了测试工作量。一方面避免了编写额外的测试程序，另一方面，也没有必要在每次运行整体测试程序时，测试这些已测试过的模块。

7.5.5 为通过测试的代码和信任的代码创建库

代码库是在不同软件组件和项目之间实现代码重用的一种手段。代码库是沉淀你的工作成果的一种手段，以便这些成果能够在将来重新发挥作用。例如，假设你花了大量精力开发了一个类，用于处理一些复杂的数值公式，专门针对公司的业务流程。事实证明，编码这些公式很容易出错，并且相比于常规意义上的代码，耗费了难以对等的测试工作量。把这个类放到代码库中，是一个很好的例子，会有很高收益：它提供了一种封装测试案例和解释复杂公式文档的手段，还潜在地避免未来重复付出这些艰辛的工作量，假如另一个应用程序中也需要使用同一个公式。

这样，在开发过程中，只要有机会，就模块化代码。这带来了更清晰的功能划分（关注

离)、更好的代码结构，以及更好的说明文档。

尽可能创建库。重构代码，以分离出业务功能中的不同方面，或者，从UI功能中分离出业务功能。这些工作都是分离库的先期准备步骤。参见第5章中的例子，以及时间服务客户端案例研究。

1.5.6 测试方面

正如在前面章节中示例和强调的，分布式系统表示的是复杂的环境，其中有很多组件以不可预知的时间顺序相互作用，还伴随着持续不断的硬件故障、消息丢失、软件组件崩溃等可能性。这为分布式应用程序的设计和开发提供了一个挑战性背景，应该对测试给予高度重视。

至关重要的是测试计划。它应该尽可能在项目分析和设计阶段就建立起来，成为项目分析和设计工作的一个组成部分。然而，在这个阶段，就能规整地描述测试活动的全集是非常困难的。因为这个集合包含的测试项，就是一个经验丰富的开发人员在其整个开发程序生命周期中展开的全体测试活动。

按照规定，测试活动通常认为是开发过程后期的重要步骤，其部分原因是，在不同的开发方法论中，将软件的生命周期描述为一系列阶段。典型的描述方式如下：分析→设计→开发→测试。

一些方法论（如螺旋模型）以一系列更短的周期重复这些步骤，但从本质上来说，每个活动块都把测试作为结尾的步骤。虽然对一个软件理论研究者来讲，这是一个有意义的序列(因为无法测试还没有开发出来的东西)，但是它没有反映真正需要做什么，才能保证高质量的产出和有效的开发过程。

对测试作用的态度以及将采取的实施方式，与构建系统的心态有关，它反映了对产出一个高质量系统的重视程度。需要把测试看作一个持续性的活动，并将测试集成到整个开发过程中。

抱着质量至上的心态，就会认为，作为开发周期的每一步，对制定出好决策还是坏决策，都是至关重要的环节。在项目刚开始时，很难概括出一个系统的完整需求并清晰地表达出来。在需求规格说明书中，不可能体现出每一个行为的细微差别，也许，这是为了要保留一些解释方面的灵活性，以确保在将来出现的任何修改都仍然不同程度地遵循了设计要求。

应该仔细检查，系统的每项需求之间，不能存在冲突，导致它们不能同时满足，而且，一旦出现了这种情况，就必须在进入设计阶段之前，全部解决掉。通过咨询系统未来的潜在用户，回答一些需要深思熟虑的问题，就能够评估需求的正确性和可行性。也可以考虑构建早期原型系统，只实现部分功能，用来检查设计理念是否与需求一致。同时，也成为用户的一种手段，看到需求规格说明书中陈述的结构，反过来重新审视需求阶段，确保它们正确反映和描述了期望的系统。

在开发过程中，应该定期测试。在测试问题上，我和学生们进行过很多讨论，他们最初的观点曾认为，在某种程度上，测试是额外的工作。实际情况是，如果持续坚持测试，实际上节省了时间和工作量，因为这些错误在经过设计阶段的传播之前，已经被发现了，并找到了产生的原因；在开发阶段越早期的时间，纠正一个糟糕的设计决策，通常代价就越要小很多。

我发现，对于一个增量式迭代测试方法，其基本前提是，在两次测试迭代过程之间，只添加或修改一个特性，这种方式最为有效。表面上看起来，这似乎是一种进展缓慢的方式，

但它的确带来了高质量的产出。因为，这能够确保，每次在做进一步的其他修改前，都充分理解了这次唯一的修订产生的效果。同时，还针对整体目标和向目标迈进的过程提供了定期反思的机会。

7.6 章末练习

7.6.1 编程练习

1. 集成NTP客户端功能到一个应用程序中。这个编程挑战与时间服务客户端使用案例相关。任务：任意选择一个合适的应用程序，为它构建NTP客户端功能。建议使用NTP库（已经作为支持材料的一部分提供）。先从学习提供的两个NTP应用的客户端程序代码（它集成了库）开始，然后在自己的应用程序中模仿库的集成方法。注意，这两个应用程序客户端，作为本练习的样例答案，已经在时间服务案例研究中讨论过。
2. 开发一套“发布－订阅”应用程序。其中，使用一个外部的ENS，作为一种解耦应用程序组件的方式。建议使用本章的第二个案例研究中提供的ENS。事件类型和值依赖于选定的应用程序主题，例如，一个分布式游戏应用程序中，事件类型可能有new-player-arrival和new-game-started。先从学习示例应用程序的程序代码开始。发布者组件和消费者组件代码已经提供在支持材料中。考虑用一种语言开发发布者程序，再用另一种语言开发消费者程序，开展异构性和互操作性方面的实验。注意，应用程序客户端已经在ENS案例研究中讨论过，可以用作本练习的样例答案。

512

7.6.2 配套资源列表

本章正文和章末练习直接参考了如下资源：

- 分布式系统版Workbench（特别是DS应用程序）。
- 资源代码。
- 可执行代码。

案例研究	程　序	可用性	说　明
1. TimeServiceClient	CaseStudy_TimeServiceClient_GUI Phase1 Monolithic	源代码可执行	在单个类中集成业务逻辑和GUI逻辑
	CaseStudy_TimeServiceClient_GUI Phase2 Refactored	源代码可执行	经过重构，业务逻辑和GUI逻辑放到不同的类
	CaseStudy_TimeServiceClient Phase3Library	源代码可执行	阶段3，TimeServiceClient库
	CaseStudy_TimeServiceClient_GUI Phase3 App-side uses library	源代码可执行	阶段3，GUI应用程序项目（静态链接TimeServiceClient库）
	CaseStudy_TimeServClnt_Console AppSide uses lib	源代码可执行	阶段3，文本应用程序项目（静态链接TimeServiceClient库）
2. 事件通知服务器	CaseStudy_Notification-Service: Server_CPP	源代码可执行	ENS服务器
	Directory service (DS)	在分布式系统版Workbench中，DS以一个应用程序提供	用来支持ENS客户端组件定位ENS

（续）

案例研究	程　　序	可用性	说　　明
2.针对测试和事件通知服务器的示例应用程序组件	CaseStudy_NotServ_Publish_Client_CPP	源代码可执行	示例应用程序发布者组件（C++版）
	CaseStudy_NotServ_Consume_Client_CPP	源代码可执行	示例应用程序消费者组件（C++版）
	CaseStudy_NotServ_Publish_Client_CSharp	源代码可执行	示例应用程序发布者组件（C#版）
	CaseStudy_NotServ_Consume_Client_CSharp	源代码可执行	示例应用程序消费者组件（C#版）
	CaseStudy_NotServ_Publish_Client_JAVA	源代码可执行	示例应用程序发布者组件（Java版）
	CaseStudy_NotServ_Consume_Client_JAVA	源代码可执行	示例应用程序消费者组件（Java版）